VULKAN

Explicit GPU programming from first principles

UNIT 1

The Explicit API

From an instance to your first compute dispatch, with nothing hidden

1.1 Instances, devices and queues

1.2 Memory, buffers and images

1.3 Commands and submission

1.4 Shaders, SPIR-V and pipelines

1.5 Descriptors and your first dispatch

1.6 Validation, debugging and a helper layer

First edition, October 2026

Contents

About this unit 2
Before you start 2
What this unit needs 2
1.1 Instances, devices and queues 3
Core ideas 3
Reading 11
Labs 12
Problem set 13
1.2 Memory, buffers and images 14
Core ideas 14
Reading 21
Labs 21
Problem set 22
1.3 Commands and submission 24
Core ideas 24
Reading 30
Labs 30
Problem set 31
1.4 Shaders, SPIR-V and pipelines 32
Core ideas 32
Reading 38
Labs 38
Problem set 40
1.5 Descriptors and your first dispatch 41
Core ideas 41
Reading 47
Labs 47
Problem set 48
1.6 Validation, debugging and a helper layer 50
Core ideas 50
Reading 56
Labs 56
Problem set 58
Sign-off 59

Vulkan, First edition, October 2026. © 2026 O'Neill Group LLC. Vulkan and the Vulkan logo are registered trademarks of the Khronos Group Inc. OpenGL and the oval logo are trademarks or registered trademarks of Hewlett Packard Enterprise in the United States and other countries, used by permission by Khronos. All other trademarks belong to their owners. This course is not endorsed by the Khronos Group.

UNIT 1

About this unit

Unit 1 builds a complete Vulkan compute program from nothing, one object at a time. It starts with the loader and an instance, chooses a device and a queue, allocates memory and binds buffers to it, and records and submits commands. It then compiles a shader to SPIR-V, builds a pipeline around it, connects the shader to buffers through descriptors, and dispatches SAXPY with the barriers it needs. The last chapter turns to the tools that find mistakes, and wraps the unit's bookkeeping in `vkf`, the small helper library that the rest of the course uses. Every program in the unit uses the raw C API, so that nothing happens that you have not written.

Chapter	You build	Hours
1.1	A device report and a logical device with validation and synchronisation validation	5
1.2	A memory map, coherent and non-coherent round trips, a sub-allocator, a format survey	6
1.3	Fill, copy and read back; staging against direct writes; the cost of a submission	5
1.4	SPIR-V read by hand, pipelines with push and specialisation constants, a pipeline cache	5
1.5	SAXPY and vector addition end to end, descriptor sets reused across dispatches	6
1.6	Six broken programs diagnosed, a capture, shader printf, your own RAII layer	5

Before you start

Complete the Handbook's setup part: the SDK, a compiler, CMake and Ninja, the companion code built, and `vkf_selftest` passing. If you have never used a GPU programming API before, read the Handbook's part on how GPUs work first.

What this unit needs

A GPU with a Vulkan 1.3 driver: any discrete or integrated GPU from NVIDIA, AMD or Intel from the last several years, or an Apple Silicon Mac through MoltenVK. The validation layers and the shader tools from the Vulkan SDK. On Windows and Linux, RenderDoc; on macOS, Xcode.

CHAPTER 1.1

Instances, devices and queues

TIME	5 hours
YOU BUILD	A program that lists every Vulkan device on your machine with its limits, features and queue families, then creates a logical device with validation and synchronisation validation switched on.
NEEDS	The Handbook's setup part completed: the companion code builds and `vkf_selftest` passes.

WHY IT MATTERS

Every Vulkan program starts the same way. It connects to the loader, chooses a GPU, and asks for a device with exactly the features and queues it intends to use, assuming nothing about the hardware. A program that enables a feature it never checked for fails on some machines and not on others, and a program that skips validation runs with bugs it cannot see. This chapter builds the start-up code that the rest of Unit 1 uses. It is also where to form two habits that save weeks later: query before you rely on anything, and run with validation from the first line.

SKIP TEST

1. What is the difference between an instance extension and a device extension, and where does each come from?
2. Why does `vkCreateInstance` fail on a Mac unless the program enables `VK_KHR_portability_enumeration`, and what must the program then do when it creates the device?
3. You want timeline semaphores and synchronization2. Which structures do you chain into `VkDeviceCreateInfo`, and what happens if the device lacks one of them?
4. A device reports one queue family with graphics, compute and transfer, one with compute and transfer, and one with transfer only. Which would you use for a background upload, and why?

If you can answer all four without looking anything up, skim the core ideas and do Lab 1.1.3 only.

Core ideas

What kind of API Vulkan is

Vulkan is a C API for graphics and compute on GPUs, published by the Khronos Group in 2016 and descended from AMD's Mantle. Version 1.1 followed in 2018, 1.2 in 2020, 1.3 in 2022 and 1.4 in 2024, and each version folded widely used extensions into the core. This course uses Vulkan 1.3, which every current desktop driver supports and which made synchronization2 and dynamic rendering part of the core.

The defining property of Vulkan is that it is *explicit*. The application creates every object the GPU uses, allocates and binds every byte of memory, records commands into buffers it submits itself, and states every ordering constraint between those commands. In exchange the driver does little that you did not ask for. It does not track resource state, insert barriers or check your calls, and it compiles shaders only when you ask it to. As a result a Vulkan program can be faster and more predictable than an OpenGL one, and a small mistake can produce a wrong image or a hang with no error at all.

Vulkan's objects are opaque handles such as `VkInstance`, `VkDevice` and `VkBuffer`. You create them with a `vkCreate*` or `vkAllocate*` call that takes a create-info structure, and destroy them with the matching `vkDestroy*` or `vkFree*` call. Almost every structure you pass in begins with two members: `sType`, which names the structure, and `pNext`, a pointer to an optional chain of further structures that extend it. Exten-

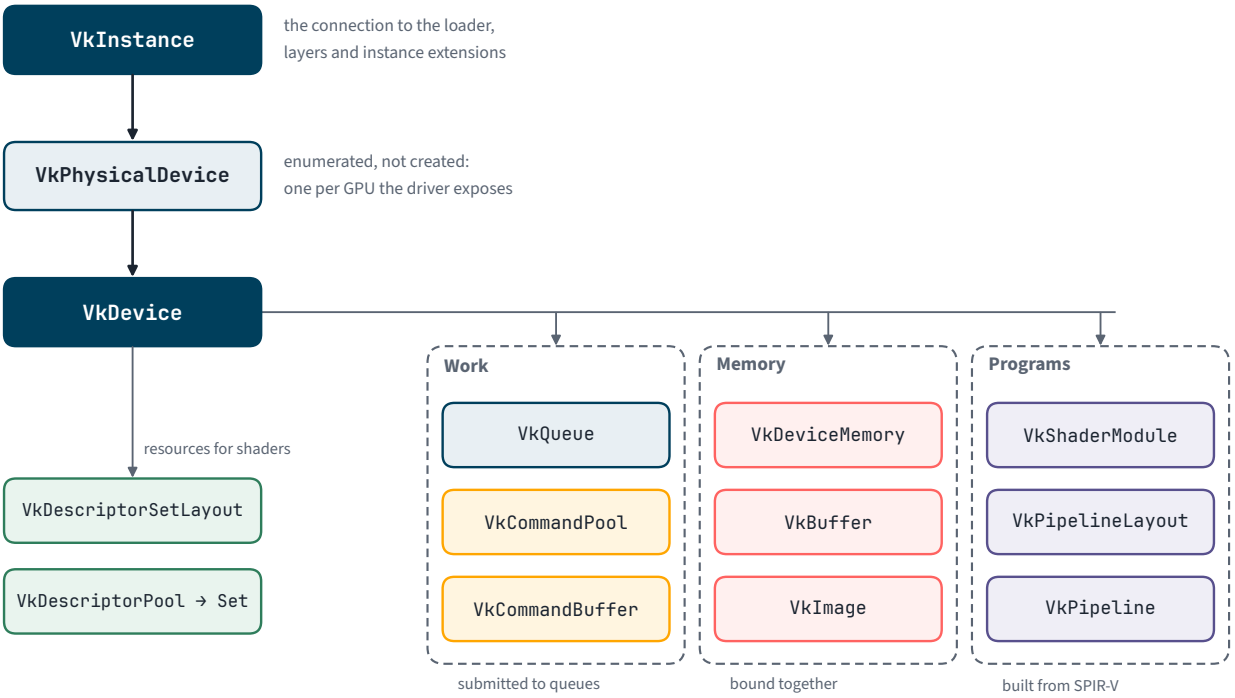


Figure 1.1.1. The objects of Unit 1. The instance and the physical device are for discovery. Everything the GPU uses is created from the logical device. Arrows point from an object to what it is used to create or reach.

sions and newer versions add capabilities by defining new structures for that chain, so existing functions keep their signatures. Figure 1.1.1 shows the objects this unit introduces.

> **COMING FROM OPENGL**
> OpenGL has one implicit object, the *context*, created through the window system (WGL, GLX, EGL or CGL, usually through GLFW or SDL) and made current on a thread. The context bundles what Vulkan separates into an instance, a device and a queue, and the operating system or driver decides which GPU it runs on. Every OpenGL call acts on the current context's global state. Vulkan has no current context and no global state: every call names the objects it uses.

The loader, layers and drivers

Your program links against the *loader*, `vulkan-1.dll` on Windows and `libvulkan` on Linux and macOS. The loader reads small JSON manifest files that describe the installed *drivers* (installable client drivers, or ICDs) and *layers*, and routes each call to the right one (Figure 1.1.2). A layer sits between your program and the driver and can observe or change calls. The most important is `VK_LAYER_KHRONOS_validation`, which checks each call against the specification's valid-usage rules and reports violations. It is installed by the Vulkan SDK and costs nothing unless you enable it.

Functionality beyond the core arrives as *extensions*. An *instance extension* adds instance-level features, such as `VK_EXT_debug_utils` for debug callbacks and object names, and comes from the loader, a layer or a driver. A *device extension* adds device-level features, such as `VK_KHR_swapchain` for presenting images, and comes from the driver of one particular GPU. You must enable every extension you use, at instance or device creation.

Creating the instance

The instance is your program's connection to the loader. Creating it means naming the Vulkan version you write against, the layers and the instance extensions you want, and optionally chaining structures that configure them. The course's start-up code lives in `code/u1/shared/basics.hpp`, which every Unit 1 program includes. It begins with error checking. Every Vulkan function that can fail returns a `VkResult`, in which negative values are errors, and the companion code turns those into exceptions that carry the result's name.

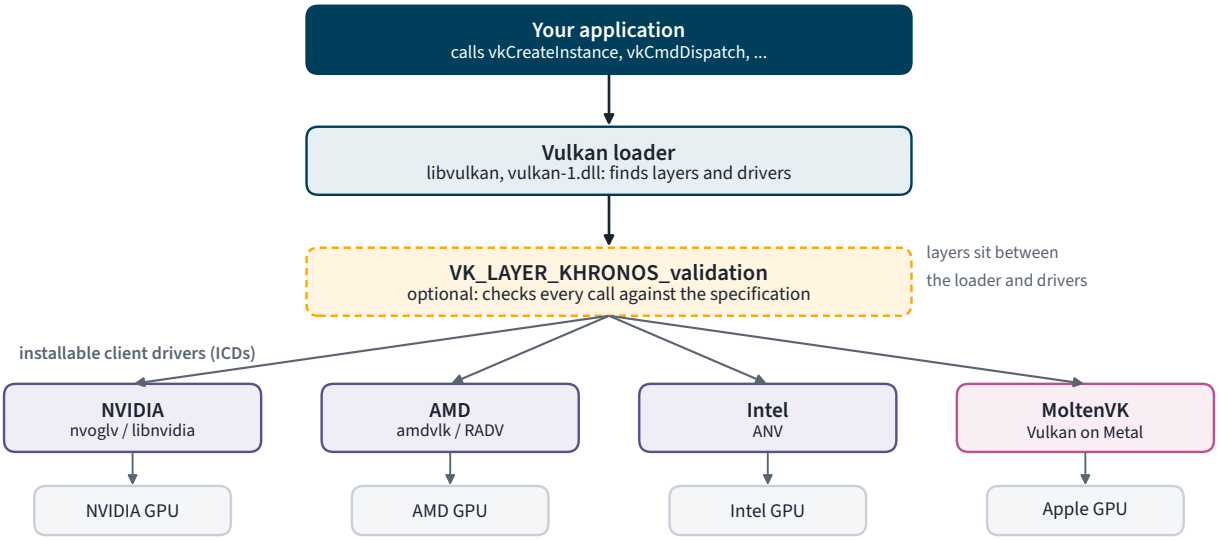


Figure 1.1.2. How a call reaches a GPU. The loader picks the driver for the device you choose and, if you enable it, sends every call through the validation layer first.

Turning a negative `VkResult` into an exception that names the call and the error — `code/u1/shared/basics.hpp`

```
inline VkResult check(VkResult result, const char* call) {
    if (result < 0) {
        throw std::runtime_error(std::string(call) + " failed: " + string_VkResult(result));
    }
    return result;
}
```

Instance creation starts by asking the loader what is installed. `vkEnumerateInstanceLayerProperties` lists the layers. Like most Vulkan queries it is called twice: once with a null array to learn the count, then again with an array of that size. The program enables validation when the layer is present, unless the environment variable `VKF_VALIDATION` is `0`.

Finding the validation layer — `code/u1/shared/basics.hpp`

```
uint32_t count = 0;
U1_CHECK(vkEnumerateInstanceLayerProperties(&count, nullptr));
std::vector<VkLayerProperties> layers(count);
U1_CHECK(vkEnumerateInstanceLayerProperties(&count, layers.data()));
const char* validationLayer = "VK_LAYER_KHRONOS_validation";
for (const VkLayerProperties& layer : layers) {
    if (std::strcmp(layer.layerName, validationLayer) == 0) {
        result.validation = validationRequested();
    }
}
```

The same two-call pattern lists the instance extensions. The program asks only for extensions that exist: `VK_KHR_portability_enumeration` when the loader offers it, which is the case on macOS (see below); `VK_EXT_debug_utils` for the messenger that reports validation messages; and `VK_EXT_layer_settings`, which lets a program configure a layer from code.

Enabling only the instance extensions the loader offers — code/u1/shared/basics.hpp

```
U1_CHECK(vkEnumerateInstanceExtensionProperties(nullptr, &count, nullptr));
std::vector<VkExtensionProperties> available(count);
U1_CHECK(vkEnumerateInstanceExtensionProperties(nullptr, &count, available.data()));
auto has = [&](const char* name) {
    for (const VkExtensionProperties& e : available) {
        if (std::strcmp(e.extensionName, name) == 0) return true;
    }
    return false;
};

std::vector<const char*> extensions;
if (has(VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME)) {
    extensions.push_back(VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME);
    result.portability = true;
}
const bool settings = result.validation && has(VK_EXT_LAYER_SETTINGS_EXTENSION_NAME);
if (result.validation) extensions.push_back(VK_EXT_DEBUG_UTILS_EXTENSION_NAME);
if (settings) extensions.push_back(VK_EXT_LAYER_SETTINGS_EXTENSION_NAME);
```

`VkApplicationInfo` names the program and, in `apiVersion`, the highest Vulkan version it is written to use. Instance creation does not fail when a driver is older than that, and the version you may use with a device is the lower of `apiVersion` and the device's own version. The settings that follow configure the validation layer. `validate_sync` turns on synchronisation validation, which is off by default because it is slower. `syncval_shader_accesses_heuristic` makes it account for the memory that shaders read and write, which it works out by analysing the SPIR-V. Without it, the version of the layer used to write this course missed a conflict between two dispatches that wrote the same buffer. Chapter 1.6 shows the layer reporting such a conflict, and Unit 3 relies on the setting.

The application info and the validation layer's settings — code/u1/shared/basics.hpp

```
const VkApplicationInfo app{
    .sType = VK_STRUCTURE_TYPE_APPLICATION_INFO,
    .pApplicationName = appName,
    .applicationVersion = 1,
    .apiVersion = VK_API_VERSION_1_3,
};

VkDebugUtilsMessengerCreateInfoEXT chained = messengerInfo();
const VkBool32 on = VK_TRUE;
const VkLayerSettingEXT syncSettings[] = {
    {validationLayer, "validate_sync", VK_LAYER_SETTING_TYPE_BOOL32_EXT, 1, &on},
    {validationLayer, "syncval_shader_accesses_heuristic",
     VK_LAYER_SETTING_TYPE_BOOL32_EXT, 1, &on},
};
const VkLayerSettingsCreateInfoEXT layerSettings{
    .sType = VK_STRUCTURE_TYPE_LAYER_SETTINGS_CREATE_INFO_EXT,
    .pNext = &chained,
    .settingCount = 2,
    .pSettings = syncSettings,
};
```

Everything comes together in `VkInstanceCreateInfo`. The `pNext` chain starts with the layer settings, which point in turn at a description of the debug messenger. A messenger chained here reports messages produced while the instance itself is being created and destroyed. The flag `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR` accompanies the portability extension.

Creating the instance — code/u1/shared/basics.hpp

```
const void* next = nullptr;
if (result.validation) {
    next = settings ? static_cast<const void*>(&layerSettings) : &chained;
}
const VkInstanceCreateInfo info{
    .sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO,
    .pNext = next,
    .flags = result.portability
                 ? VkInstanceCreateFlags(VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR)
                 : 0u,
    .pApplicationInfo = &app,
    .enabledLayerCount = result.validation ? 1u : 0u,
    .ppEnabledLayerNames = &validationLayer,
    .enabledExtensionCount = static_cast<uint32_t>(extensions.size()),
    .ppEnabledExtensionNames = extensions.data(),
};
U1_CHECK(vkCreateInstance(&info, nullptr, &result.instance));
```

Once the instance exists, the program creates a second messenger from the same description, which reports everything between creation and destruction. `vkCreateDebugUtilsMessengerEXT` belongs to an extension, and the loader does not export most extension functions from its library, so the program asks the instance for its address with `vkGetInstanceProcAddr`.

The messenger for the instance's lifetime — code/u1/shared/basics.hpp

```
if (result.validation) {
    auto create = reinterpret_cast<PFN_vkCreateDebugUtilsMessengerEXT>(
        vkGetInstanceProcAddr(result.instance, "vkCreateDebugUtilsMessengerEXT"));
    const VkDebugUtilsMessengerCreateInfoEXT standalone = messengerInfo();
    U1_CHECK(create(result.instance, &standalone, nullptr, &result.messenger));
}
```

The messenger's callback counts errors, so that every program can fail when validation reports one, and prints each message. Returning `VK_FALSE` tells the layer to let the call proceed.

The debug messenger: count errors, print everything — code/u1/shared/basics.hpp

```
inline VKAPI_ATTR VkBool32 VKAPI_CALL onValidationMessage(
    VkDebugUtilsMessageSeverityFlagBitsEXT severity, VkDebugUtilsMessageTypeFlagsEXT,
    const VkDebugUtilsMessengerCallbackDataEXT* data, void*) {
    const bool error = (severity & VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT) != 0;
    if (error) ++validationErrors;
    std::fprintf(stderr, "[validation %s] %s\n", error ? "error" : "warning", data->pMessage);
    return VK_FALSE;
}

inline VkDebugUtilsMessengerCreateInfoEXT messengerInfo() {
    return {
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT,
        .messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT |
                           VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT,
        .messageType = VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
                       VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT |
                       VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT,
        .pfnUserCallback = onValidationMessage,
    };
}
```

> **KEY IDEA**
> Keep the validation layer on throughout development. It is often the only sign that a program is broken, because invalid Vulkan usage is undefined behaviour: it may work on your GPU and fail on the next one, or work today and fail after a driver update. Every program in this course runs with validation on and fails if the layer reports an error.

Portability and macOS

Apple's GPUs do not have native Vulkan drivers. MoltenVK implements Vulkan on top of Metal and is installed with the Vulkan SDK. Because Metal cannot express every Vulkan behaviour, MoltenVK is a *portability* implementation: conformant for everything it supports, but missing a few features that a full implementation must have. Since 2022 the loader hides portability drivers from programs that have not declared that they support them. To see MoltenVK you must enable `VK_KHR_portability_enumeration` and set `VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR`, as the code above does. Without both, on a Mac whose only driver is MoltenVK, `vkCreateInstance` finds no driver at all and fails with `VK_ERROR_INCOMPATIBLE_DRIVER`. A device that offers the `VK_KHR_portability_subset` extension must then have it enabled when the logical device is created, and the extension's feature structure lists what is missing.

Physical devices

A *physical device* is a GPU as the driver presents it. You do not create physical devices. You enumerate them, and they live as long as the instance. Each one reports three kinds of information, all of which a careful program reads before relying on anything:

- **Properties**, from `vkGetPhysicalDeviceProperties`: the name, vendor, device type, the highest Vulkan version supported, and the *limits*, more than a hundred numbers such as the largest workgroup or the smallest alignment for a buffer offset.
- **Features**, from `vkGetPhysicalDeviceFeatures2`: optional capabilities such as 64-bit floats in shaders or timeline semaphores, each a `VkBool32`. The Vulkan 1.1, 1.2 and 1.3 features are reported by chaining one structure per version into `pNext`.
- **Queue families**, from `vkGetPhysicalDeviceQueueFamilyProperties`: groups of queues with the same capabilities, described next.

Reading features through a pNext chain: one call fills all three structures — code/u1/devices/main.cpp

```
void printFeatures(VkPhysicalDevice device) {
    VkPhysicalDeviceVulkan13Features v13{
        .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_3_FEATURES};
    VkPhysicalDeviceVulkan12Features v12{
        .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_2_FEATURES, .pNext = &v13};
    VkPhysicalDeviceFeatures2 features{.sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2,
                                       .pNext = &v12};
    vkGetPhysicalDeviceFeatures2(device, &features);

    std::printf(
        "    synchronization2 %s, dynamicRendering %s, timelineSemaphore %s, "
        "bufferDeviceAddress %s\n",
        v13.synchronization2 ? "yes" : "no", v13.dynamicRendering ? "yes" : "no",
        v12.timelineSemaphore ? "yes" : "no", v12.bufferDeviceAddress ? "yes" : "no");
    std::printf("    shaderFloat64 %s, shaderInt64 %s, pipelineStatisticsQuery %s\n",
                features.features.shaderFloat64 ? "yes" : "no",
                features.features.shaderInt64 ? "yes" : "no",
                features.features.pipelineStatisticsQuery ? "yes" : "no");
}
```

Choosing a device is a policy decision. The course's code prefers a discrete GPU, then an integrated one, and requires Vulkan 1.3. Production code would also check that every feature and extension it needs is supported, and might let the user override the choice.

Picking a device: Vulkan 1.3 at least, discrete GPUs first — code/u1/shared/basics.hpp

```
inline VkPhysicalDevice pickPhysicalDevice(VkInstance instance) {
    uint32_t count = 0;
    U1_CHECK(vkEnumeratePhysicalDevices(instance, &count, nullptr));
    std::vector<VkPhysicalDevice> devices(count);
    U1_CHECK(vkEnumeratePhysicalDevices(instance, &count, devices.data()));

    VkPhysicalDevice best = VK_NULL_HANDLE;
    int bestScore = -1;
    for (VkPhysicalDevice device : devices) {
        VkPhysicalDeviceProperties properties;
```

```
        vkGetPhysicalDeviceProperties(device, &properties);
        if (properties.apiVersion < VK_API_VERSION_1_3) continue;
        int score = 0;
        if (properties.deviceType == VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU) {
            score = 3;
        } else if (properties.deviceType == VK_PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU) {
            score = 2;
        }
        if (score > bestScore) {
            best = device;
            bestScore = score;
        }
    }
    if (best == VK_NULL_HANDLE) throw std::runtime_error("no device supports Vulkan 1.3");
    return best;
}
```

Queues and queue families

A *queue* is where you submit work, and a GPU runs work from several queues at once. Queues come in *families*, each with a set of capabilities: graphics, compute, transfer (copies), and some rarer ones such as sparse binding and video. Every family that supports graphics or compute also supports transfer commands, whether or not it sets the transfer flag.

A typical discrete GPU exposes one family that does everything, with one or more queues; a compute-only family with several queues, which runs alongside graphics (*async compute*, Chapter 3.5); and a transfer-only family that drives the copy engines, ideal for uploads that should not disturb rendering. The exact arrangement varies by vendor and driver, which is why programs search for the family they need instead of assuming an index. Apple's GPU, through MoltenVK, reports four identical families of one queue each:

The device report from Lab 1.1.1 on an Apple M2 Pro

```
$ ./build/bin/u1_devices
loader supports Vulkan 1.4.363
validation on, portability enumeration enabled
device 0: Apple M2 Pro (integrated GPU)
    Vulkan 1.3.357
    maxComputeWorkGroupSize 1024 x 1024 x 1024, maxComputeWorkGroupInvocations 1024
    maxComputeSharedMemorySize 32768 bytes, maxPushConstantsSize 4096 bytes
    maxStorageBufferRange 4294967295 bytes, maxMemoryAllocationCount 1073741824
    timestampPeriod 1.00 ns
    synchronization2 yes, dynamicRendering yes, timelineSemaphore yes, bufferDeviceAddress yes
    shaderFloat64 no, shaderInt64 yes, pipelineStatisticsQuery no
    family 0: 1 queue(s) graphics compute transfer, 64 timestamp bits
    family 1: 1 queue(s) graphics compute transfer, 64 timestamp bits
    family 2: 1 queue(s) graphics compute transfer, 64 timestamp bits
    family 3: 1 queue(s) graphics compute transfer, 64 timestamp bits
created a device on Apple M2 Pro with queue family 0
```

The first family that has every required capability — code/u1/shared/basics.hpp

```
inline uint32_t findQueueFamily(VkPhysicalDevice device, VkQueueFlags required) {
    uint32_t count = 0;
    vkGetPhysicalDeviceQueueFamilyProperties(device, &count, nullptr);
    std::vector<VkQueueFamilyProperties> families(count);
    vkGetPhysicalDeviceQueueFamilyProperties(device, &count, families.data());
    for (uint32_t i = 0; i < count; ++i) {
        if ((families[i].queueFlags & required) == required) return i;
    }
    throw std::runtime_error("no queue family has the required capabilities");
}
```

The logical device

The *logical device*, `VkDevice`, is your program's session with one physical device. Creating it is where you commit to what you will use:

- **Queues.** Which queues, from which families, each with a priority between 0 and 1 that the driver may use to schedule between them.
- **Features.** Which features to enable. Features you do not enable may not be used, even if the hardware has them, and asking for one the device lacks makes creation fail with `VK_ERROR_FEATURE_NOT_PRESENT`.
- **Device extensions.** Which device extensions to enable, including `VK_KHR_portability_subset` where the device offers it.

The course enables three Vulkan 1.3 and 1.2 features from the start. *Synchronization2* is the modern form of barriers and submission that Unit 3 teaches. *Timeline semaphores* appear in Chapter 3.4. *Maintenance4* allows, among other things, the `LocalSizeId` execution mode, which the shader compiler emits for the workgroup sizes set by specialisation constants in Chapter 1.4. All three are required to be supported by any Vulkan 1.3 device, so enabling them without checking is safe once the device's version is known.

Each queue the program wants is described by a `VkDeviceQueueCreateInfo`: a family, a count and one priority per queue. Unit 1 needs a single queue that can run compute and graphics work.

One queue from the chosen family — code/u1/shared/basics.hpp

```
const float priority = 1.0f;
const VkDeviceQueueCreateInfo queueInfo{
    .sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO,
    .queueFamilyIndex = result.queueFamily,
    .queueCount = 1,
    .pQueuePriorities = &priority,
};
```

Features are enabled with the same structures that reported them, chained into `pNext`. Only the members set to `VK_TRUE` are enabled.

Enabling exactly three features — code/u1/shared/basics.hpp

```
VkPhysicalDeviceVulkan13Features v13{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_3_FEATURES,
    .synchronization2 = VK_TRUE,
    .maintenance4 = VK_TRUE,
};
VkPhysicalDeviceVulkan12Features v12{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_2_FEATURES,
    .pNext = &v13,
    .timelineSemaphore = VK_TRUE,
};
const VkPhysicalDeviceFeatures2 features{
    .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2, .pNext = &v12};
```

The device extensions are found with the two-call pattern again. The only one Unit 1 needs is `VK_KHR_portability_subset`, which must be enabled whenever the device offers it.

Enabling the portability subset where it exists, then creating the device — code/u1/shared/basics.hpp

```
U1_CHECK(vkEnumerateDeviceExtensionProperties(physical, nullptr, &count, nullptr));
std::vector<VkExtensionProperties> available(count);
U1_CHECK(
    vkEnumerateDeviceExtensionProperties(physical, nullptr, &count, available.data()));
std::vector<const char*> extensions;
for (const VkExtensionProperties& e : available) {
    if (std::strcmp(e.extensionName, "VK_KHR_portability_subset") == 0) {
        extensions.push_back("VK_KHR_portability_subset");
    }
}

const VkDeviceCreateInfo info{
    .sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO,
    .pNext = &features,
    .queueCreateInfoCount = 1,
    .pQueueCreateInfos = &queueInfo,
    .enabledExtensionCount = static_cast<uint32_t>(extensions.size()),
    .ppEnabledExtensionNames = extensions.data(),
};
U1_CHECK(vkCreateDevice(physical, &info, nullptr, &result.device));
vkGetDeviceQueue(result.device, result.queueFamily, 0, &result.queue);
```

Queues are not created separately. `vkGetDeviceQueue` returns the queues that device creation made. Objects are destroyed in reverse order of creation: everything made from the device, then the device, then the debug messenger and the instance. The validation layer reports anything you forget, as Chapter 1.6 demonstrates.

PITFALL

It is tempting to query features into a `VkPhysicalDeviceFeatures2` chain and pass the same chain to `vkCreateDevice`. That enables every feature the device supports, including ones with a cost: `robustBufferAccess`, for example, adds a bounds check to every buffer access in every shader. Keep the queried structures and the enabled structures separate, and enable only what you use.

COMING FROM OPENGL

OpenGL reports capabilities through `glGetString(GL_RENDERER)`, `glGetIntegerv` for limits such as `GL_MAX_COMPUTE_WORK_GROUP_SIZE`, and `glGetStringi(GL_EXTENSIONS, i)` for extensions. Every extension the driver supports is available as soon as the context exists, with nothing to enable. OpenGL also checks every call as part of the specification and records failures for `glGetError`, which costs time on every call, even in a finished program. Vulkan moves that checking into a layer that you enable while developing and leave out when you ship.

Reading

- The Vulkan specification, chapters “Initialization”, “Devices and Queues” and “Extending Vulkan” (docs.vulkan.org/spec). Read the sections on `vkCreateInstance` and `vkCreateDevice` with their valid-usage lists.
- The Vulkan Guide (docs.vulkan.org/guide): “Loader”, “Layers”, “pNext and sType”, “Querying Properties, Extensions, Features, Limits, and Formats” and “Portability Initiative”.
- The output of `vulkaninfo` for your own GPU, read alongside the specification’s “Limits” chapter.

Labs

LAB 1.1.1 Run the device report — 45 minutes

Goal. Build Unit 1, run the device report, and check it against `vulkaninfo`.

Code. `code/u1/devices`

1. Build the companion code as the Handbook describes, then run `build/bin/u1_devices` from the code folder. On macOS with the Homebrew validation layer, export `DYLD_LIBRARY_PATH` first, as the Handbook explains.
2. Run `vulkaninfo --summary` and compare the device name, API version, and the number of queue families with the report.
3. Run the report with `VKF_VALIDATION=0` and note the first line that changes.

DONE WHEN

- [ ] The program exits with status 0 and prints `created a device`.
- [ ] Every number in the report agrees with `vulkaninfo`.

EVIDENCE FOR THE NOTEBOOK

- The full report, saved to your notebook.
- A sentence on what the queue families tell you about your GPU.

LAB 1.1.2 Break the instance on purpose — 45 minutes

Goal. See how creation fails, and what each failure means.

Code. `code/u1/devices`

1. In `createInstance`, push the name of an extension that does not exist, such as `"VK_EXT_not_real"`, onto `extensions` just before the create-info is filled in. Run the program and record the error it reports.
2. Undo that. Now set `enabledLayerCount` to 1 unconditionally and point `ppEnabledLayerNames` at a misspelt name such as `"VK_LAYER_KHRONOS_validaton"`. Record the result.
3. Undo that. On macOS, leave the extension list alone but set `flags` to 0, so that portability enumeration is enabled but not requested. Record the result. On other systems, explain what you would expect.
4. Restore the file from the original code archive and rebuild.

DONE WHEN

- [ ] You have recorded `VK_ERROR_EXTENSION_NOT_PRESENT` and `VK_ERROR_LAYER_NOT_PRESENT`, and can say which check in `createInstance` would have prevented each.
- [ ] On macOS, you have recorded `VK_ERROR_INCOMPATIBLE_DRIVER` and can explain it in one sentence.

EVIDENCE FOR THE NOTEBOOK

- Each change and the error it produced.

LAB 1.1.3 Read more of the feature chain — 1.5 hours

Goal. Extend the report with the properties a compute programmer needs.

Code. `code/u1/devices`

1. Chain `VkPhysicalDeviceSubgroupProperties` into `vkGetPhysicalDeviceProperties2` and print the subgroup size, the supported stages and the supported operations.
2. Chain `VkPhysicalDeviceVulkan11Features` and print `storageBuffer16BitAccess` and `shaderDrawParameters`.
3. Print `maxComputeWorkGroupCount` and `minStorageBufferOffsetAlignment` from the limits.

DONE WHEN

- [ ] The new lines match `vulkaninfo` for your device.
- [ ] The program still exits 0 with no validation messages.

EVIDENCE FOR THE NOTEBOOK

- The extended output, and the subgroup size, which Chapter 2.1 relies on.

LAB 1.1.4 Ask for something the device lacks — 45 minutes

Goal. Learn what happens when a program assumes a feature.

Code. `code/u1/devices`

1. Find a feature your device does not support in the report, such as `shaderFloat64` on Apple GPUs or `pipelineStatisticsQuery` where it is absent.
2. Enable it in `createDevice` by setting the member in `features.features` (make `features` non-const) and run. Record the result, and the message in which the validation layer names the missing feature.
3. Change `createDevice` so that it checks support first and enables the feature only when present, printing which way it went.

DONE WHEN

- [ ] The unchecked version fails with `VK_ERROR_FEATURE_NOT_PRESENT`, or, if your device supports every candidate, you have explained why from the report.
- [ ] The checked version runs cleanly on your device.

EVIDENCE FOR THE NOTEBOOK

- The failing and the fixed code, and the output of each.

Problem set

1. Explain why physical devices are enumerated and logical devices are created. What would go wrong if an application could create a physical device?
2. A program sets `VkApplicationInfo::apiVersion` to `VK_API_VERSION_1_2` and runs on a device that reports 1.3. May it call `vkCmdPipelineBarrier2`? If not, what would it have to do to use synchronization2 on that device? Find the passage in the specification that decides this.
3. A device has families G|C|T (1 queue), C|T (2 queues) and T (2 queues). Write the `VkDeviceQueueCreateInfo` array for a program that wants one queue for rendering, one for async compute and one for uploads.
4. Why must the debug messenger be destroyed before the instance? What does the validation layer report if you forget?
5. A colleague passes the `VkPhysicalDeviceFeatures2` chain they used to query support straight into `vkCreateDevice`. Name two consequences, one for correctness and one for performance.

COMPLETION CHECKLIST

- [ ] I can create an instance with validation, synchronisation validation and portability enumeration, and say what each part does.
- [ ] I can enumerate physical devices and read their properties, limits, features and queue families.
- [ ] I can create a logical device that enables only the features and extensions I use.
- [ ] Lab 1.1.1–1.1.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.2

Memory, buffers and images

TIME	6 hours
YOU BUILD	A memory-map printer, a round trip through mapped memory with the flush that non-coherent memory needs, a linear sub-allocator that places a thousand buffers in one allocation, and a survey of image formats and their memory.
NEEDS	Chapter 1.1.

WHY IT MATTERS

In OpenGL you asked for a buffer or a texture and the driver decided where it lived, when it moved, and how the CPU's writes reached the GPU. Vulkan hands every one of those decisions to you. The GPU's memory is described as a small set of heaps and types, resources are created without memory and bound to it afterwards, and the CPU's writes reach the GPU only under rules you must follow. A poor choice can make an upload run far below the bus's full speed, or make a program fail on some drivers when it makes its four-thousandth allocation.

SKIP TEST

1. What is the difference between a memory heap and a memory type? Which property flag tells you that the CPU can map memory, and which tells you that it need not flush?
2. A buffer's memory requirements are size 1000, alignment 256 and `memoryTypeBits` `0b0110`. Which memory types may back it, and at which offsets in an allocation may it be bound?
3. Why do Vulkan programs allocate a few large blocks of device memory and place many resources in each?
4. What does an image's tiling decide, and why can you not map an optimally tiled image and write its pixels?
5. When must a program call `vkFlushMappedMemoryRanges`, and when `vkInvalidateMappedMemoryRanges`?

If you can answer all five, skim the core ideas and do Labs 1.2.2 and 1.2.3.

Core ideas

Where memory lives

A discrete GPU has its own memory, VRAM, on the graphics card, and reaches the system's RAM across the PCI Express bus. The GPU reads its VRAM at hundreds of gigabytes per second and system RAM at a few tens at best. An integrated GPU, including Apple's, shares one pool of memory with the CPU. Vulkan describes both arrangements in the same terms.

A *memory heap* is a physical pool of memory with a size and, if the device reads it fast, the flag `VK_MEMORY_HEAP_DEVICE_LOCAL_BIT`. A *memory type* is a way of using one heap, described by property flags:

- `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT`: the device reads and writes this memory at full speed.
- `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`: the CPU can map it into its address space with `vkMapMemory`.
- `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT`: the CPU's writes and the device's writes reach each other without explicit flush and invalidate calls (see below).
- `VK_MEMORY_PROPERTY_HOST_CACHED_BIT`: the CPU caches this memory. Reads by the CPU are fast; without this flag the CPU reads mapped memory uncached, which is many times slower.

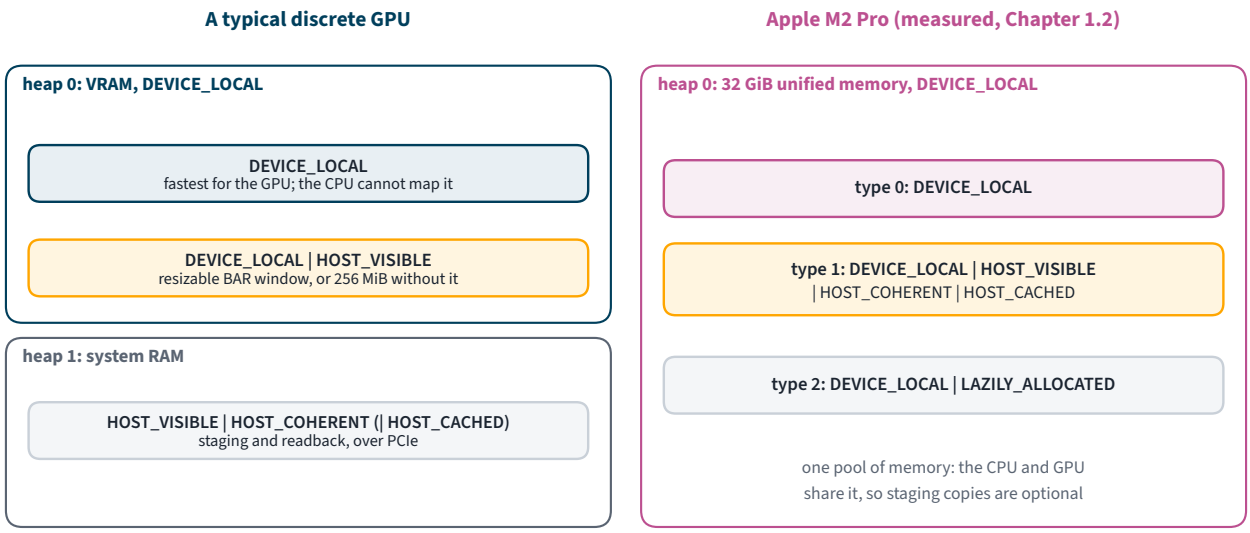


Figure 1.2.1. Heaps and memory types on a typical discrete GPU and on the Apple M2 Pro. Each type is a way of using a heap, and its flags say who can reach the memory and how fast.

- `VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT`: the device may never back this memory with real storage. Tile-based GPUs, Apple's among them, use it for render targets that live only in on-chip memory (Unit 4).

Figure 1.2.1 compares a typical discrete GPU with the Apple M2 Pro used to write this course. The discrete GPU offers memory the CPU cannot touch, memory in system RAM that both can use, and often a type that is both device-local and host-visible: the whole of VRAM when the system enables resizable BAR, or a 256 MiB window when it does not. The M2 Pro has one heap and a type that is device-local, host-visible, coherent and cached at once.

A program reads the layout with `vkGetPhysicalDeviceMemoryProperties`. The memory-map printer of Lab 1.2.1 prints both arrays:

Printing the heaps and memory types — code/u1/memory/main.cpp

```
void printMemory(const VkPhysicalDeviceMemoryProperties& memory) {
    for (uint32_t i = 0; i < memory.memoryHeapCount; ++i) {
        const VkMemoryHeap& heap = memory.memoryHeaps[i];
        std::printf("heap %u: %.1f GiB%s\n", i, double(heap.size) / (1u << 30),
                    (heap.flags & VK_MEMORY_HEAP_DEVICE_LOCAL_BIT) ? " device-local" : "");
    }
    for (uint32_t i = 0; i < memory.memoryTypeCount; ++i) {
        const VkMemoryType& type = memory.memoryTypes[i];
        std::printf("type %u: heap %u,%s\n", i, type.heapIndex,
                    memoryFlags(type.propertyFlags).c_str());
    }
}
```

u1_memory on an Apple M2 Pro: one heap, three types, and the results of the chapter's experiments

```
$ ./build/bin/u1_memory
Apple M2 Pro
heap 0: 32.0 GiB device-local
type 0: heap 0, DEVICE_LOCAL
type 1: heap 0, DEVICE_LOCAL HOST_VISIBLE HOST_COHERENT HOST_CACHED
type 2: heap 0, DEVICE_LOCAL LAZILY_ALLOCATED
mapped 4194304 bytes in memory type 1 (coherent, no flush): ok
1000 buffers, 31.4 MiB requested, in one allocation: 31.5 MiB used, no overlaps
optimal 1024 x 1024 RGBA8 image: 4194304 bytes, alignment 128 (pixels: 4194304)
VK_FORMAT_R8G8B8A8_UNORM       storage yes sampled yes colour attachment yes
VK_FORMAT_R8G8B8A8_SRGB        storage yes sampled yes colour attachment yes
VK_FORMAT_R32_SFLOAT           storage yes sampled yes colour attachment yes
VK_FORMAT_R16G16B16A16_SFLOAT  storage yes sampled yes colour attachment yes
```

> **COMING FROM OPENGL**
> `glBufferData` allocates storage, and the driver chooses where it lives from a usage hint such as `GL_STATIC_DRAW` or `GL_STREAM_DRAW`. It may move the storage later if your usage turns out to differ from the hint. OpenGL 4.4's `glBufferStorage` with `GL_MAP_PERSISTENT_BIT` and `GL_MAP_COHERENT_BIT` is the closest relative of a persistently mapped `HOST_VISIBLE | HOST_COHERENT` allocation in Vulkan. Vulkan never moves memory out of the type you chose.

Buffers and their memory requirements

A *buffer* is a range of bytes that commands and shaders can use. Creating one with `vkCreateBuffer` allocates no memory. The create-info states the size and the *usage flags*: how the buffer will be used, such as `VK_BUFFER_USAGE_STORAGE_BUFFER_BIT` for shader reads and writes, `VK_BUFFER_USAGE_TRANSFER_SRC_BIT` and `VK_BUFFER_USAGE_TRANSFER_DST_BIT` for copies, or `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT` for indirect commands. Using a buffer in a way its usage flags do not include is invalid, and the driver may choose a layout or a memory type based on them. The sharing mode decides whether one queue family owns the buffer at a time (`VK_SHARING_MODE_EXCLUSIVE`, the default and the faster choice) or several may use it at once (`VK_SHARING_MODE_CONCURRENT`). Chapter 3.5 explains the difference.

`vkGetBufferMemoryRequirements` then reports what the buffer needs from memory: a size, which may be larger than the size you asked for; an alignment, which the offset you bind it at must be a multiple of; and `memoryTypeBits`, a bit mask in which bit i is set when memory type i may back the buffer. A program picks the first type that is both allowed and has the properties it wants:

Choosing a memory type: allowed by the resource and with every required property — code/u1/shared/basics.hpp

```
inline uint32_t findMemoryType(const VkPhysicalDeviceMemoryProperties& memory,
                               uint32_t allowedTypes, VkMemoryPropertyFlags required) {
    for (uint32_t i = 0; i < memory.memoryTypeCount; ++i) {
        const bool allowed = (allowedTypes & (1u << i)) != 0;
        const bool suitable = (memory.memoryTypes[i].propertyFlags & required) == required;
        if (allowed && suitable) return i;
    }
    throw std::runtime_error("no memory type has the required properties");
}
```

The specification constrains the order of the memory types: a type whose flags are a strict subset of another's comes first, and of two types with the same flags, the one on the faster heap comes first. Taking the first suitable type therefore takes the one with the fewest extra properties, which is usually the cheapest that does the job.

Unit 1's `createBuffer` performs every step for one buffer: create, query requirements, allocate, bind and, for host-visible memory, map.

A buffer with its own allocation — code/u1/shared/basics.hpp

```
inline Buffer createBuffer(const Device& device, VkDeviceSize size, VkBufferUsageFlags usage,
                           VkMemoryPropertyFlags properties) {
    Buffer result;
    result.size = size;
    const VkBufferCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
        .size = size,
        .usage = usage,
        .sharingMode = VK_SHARING_MODE_EXCLUSIVE,
    };
    U1_CHECK(vkCreateBuffer(device.device, &info, nullptr, &result.buffer));

    VkMemoryRequirements requirements;
    vkGetBufferMemoryRequirements(device.device, result.buffer, &requirements);
    result.memoryType = findMemoryType(device.memory, requirements.memoryTypeBits, properties);
    const VkMemoryAllocateInfo allocation{
        .sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
        .allocationSize = requirements.size,
```

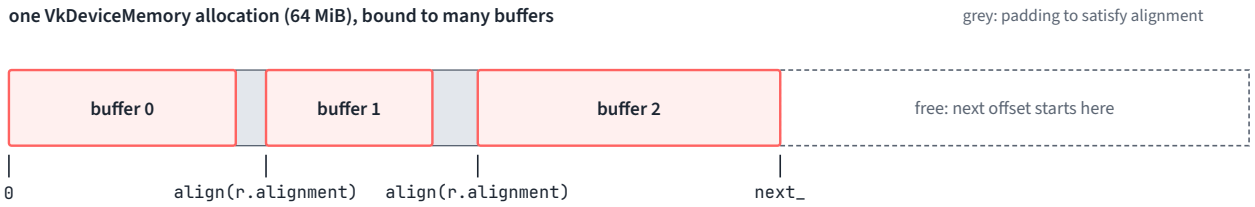


Figure 1.2.2. A linear sub-allocator. Each buffer is placed at the next offset rounded up to its alignment, and the grey gaps are the padding that alignment costs.

```
        .memoryTypeIndex = result.memoryType,
    };
    U1_CHECK(vkAllocateMemory(device.device, &allocation, nullptr, &result.memory));
    U1_CHECK(vkBindBufferMemory(device.device, result.buffer, result.memory, 0));

    if (properties & VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT) {
        U1_CHECK(
            vkMapMemory(device.device, result.memory, 0, VK_WHOLE_SIZE, 0, &result.mapped));
    }
    return result;
}
```

Allocations are expensive and limited

`createBuffer` suits a handful of buffers, but it does not scale to a real program. `vkAllocateMemory` asks the operating system's kernel driver for memory. A call can take a millisecond or more, and the number of live allocations is limited by `maxMemoryAllocationCount`. The specification guarantees only 4096, and several desktop drivers report exactly that. Apple's driver reports more than a billion. Allocations are also made in coarse units, often 4 or 64 KiB, so a small buffer in its own allocation wastes most of it.

Programs avoid these costs with *sub-allocation*: they allocate a few large blocks, typically 64 to 256 MiB each, and bind many resources at different offsets within them (Figure 1.2.2). `vkBindBufferMemory` takes the offset as an argument, and the only rules are the ones the memory requirements state: the type must be allowed, the offset must be a multiple of the alignment, and the resource must fit.

The simplest sub-allocator only moves forward. It cannot free a single buffer, only the whole block, which suits resources that are created and destroyed together, such as everything a frame or a level needs.

A linear sub-allocator: one allocation, many buffers — code/u1/memory/main.cpp

```
class LinearAllocator {
public:
    LinearAllocator(VkDevice device, VkDeviceSize capacity, uint32_t memoryType)
        : device_(device), capacity_(capacity), type_(memoryType) {
        const VkMemoryAllocateInfo info{
            .sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
            .allocationSize = capacity,
            .memoryTypeIndex = memoryType,
        };
        U1_CHECK(vkAllocateMemory(device, &info, nullptr, &memory_));
    }
    ~LinearAllocator() { vkFreeMemory(device_, memory_, nullptr); }
    LinearAllocator(const LinearAllocator&) = delete;
    LinearAllocator& operator=(const LinearAllocator&) = delete;

    VkDeviceSize bind(VkBuffer buffer) {
        VkMemoryRequirements r;
        vkGetBufferMemoryRequirements(device_, buffer, &r);
        if ((r.memoryTypeBits & (1u << type_)) == 0) {
            throw std::runtime_error("buffer cannot use this memory type");
        }
        const VkDeviceSize offset = (next_ + r.alignment - 1) / r.alignment * r.alignment;
```

```
        if (offset + r.size > capacity_) throw std::runtime_error("the block is full");
        U1_CHECK(vkBindBufferMemory(device_, buffer, memory_, offset));
        next_ = offset + r.size;
        return offset;
    }
    VkDeviceSize used() const { return next_; }

private:
    VkDevice device_;
    VkDeviceMemory memory_ = VK_NULL_HANDLE;
    VkDeviceSize capacity_;
    VkDeviceSize next_ = 0;
    uint32_t type_;
};
```

The memory experiment above places a thousand buffers of random sizes between 256 bytes and 64 KiB in one 64 MiB allocation and checks that none overlaps the next. Rounding and alignment cost about 0.1 MiB of the 31.5 MiB used.

> **PITFALL**
> When a block holds both buffers and optimally tiled images, a further limit applies: `bufferImageGranularity`. A linear resource (a buffer, or a linear image) and an optimal image that share a page of that size may interfere with each other on some hardware. Allocators either keep the two kinds in separate blocks or round their offsets apart. The linear allocator above places only buffers, so the limit does not arise.

Production allocators also free individual resources, keep separate blocks per memory type, give large resources an allocation of their own, and report statistics. Writing one well is a project in itself, and most programs use AMD's open-source Vulkan Memory Allocator, which Chapter 4.6 discusses.

> **KEY IDEA**
> Creating a resource and giving it memory are separate steps, and every allocator is built on that separation. It lets one allocation serve thousands of resources, and it lets two resources that are never used at the same time share memory.

Mapping and coherence

`vkMapMemory` returns a CPU pointer into a host-visible allocation. The usual practice is to map once, keep the pointer for the allocation's lifetime and write through it whenever needed. Unit 1's `createBuffer` does this. An allocation can only be mapped once at a time, which is one more reason to share mapped blocks between resources.

The CPU's caches sit between your writes and the memory. With `HOST_COHERENT` memory the hardware keeps the two in step, so once your writes are complete the device can see them. Without it, your writes may still be in a CPU cache, and you must *flush* the range you wrote with `vkFlushMappedMemoryRanges` before the device reads it. In the other direction, after the device writes non-coherent memory, you must *invalidate* the range with `vkInvalidateMappedMemoryRanges` before reading it, or the CPU may read stale cached values. Ranges passed to either call must be aligned to the device's `nonCoherentAtomSize`, or cover the whole allocation.

Flushing host writes only when the memory is not coherent — code/u1/memory/main.cpp

```
bool flushHostWrites(const u1::Device& device, const u1::Buffer& buffer) {
    const VkMemoryType& type = device.memory.memoryTypes[buffer.memoryType];
    if (type.propertyFlags & VK_MEMORY_PROPERTY_HOST_COHERENT_BIT) return false;
    const VkMappedMemoryRange range{
        .sType = VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE,
        .memory = buffer.memory,
        .offset = 0,
        .size = VK_WHOLE_SIZE,
    };
    U1_CHECK(vkFlushMappedMemoryRanges(device.device, 1, &range));
    return true;
}
```

The round trip below asks only for `HOST_VISIBLE` memory, so it may receive a non-coherent type on some devices and must handle both. On the M2 Pro it receives type 1, which is coherent.

Writing through a persistent mapping — code/u1/memory/main.cpp

```
const VkDeviceSize bytes = 4 << 20;
u1::Buffer buffer = u1::createBuffer(device, bytes, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT,
                                     VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT);
auto* words = static_cast<uint32_t*>(buffer.mapped);
for (uint32_t i = 0; i < bytes / 4; ++i) words[i] = i * 2654435761u;
const bool flushed = flushHostWrites(device, buffer);
```

Coherence makes writes reach memory, but it does not order them against the device's work. The device must not read data the CPU is still writing, and the CPU must not read results the device has not finished. Two rules handle this, and both appear again in Chapter 1.3 and Unit 3. Host writes made before `vkQueueSubmit`, and flushed if the memory is not coherent, are visible to the commands of that submission. Device writes become available to the host only after a barrier to the host and a wait for the work to finish.

> **PITFALL**
> Memory without `HOST_CACHED` is often *write-combined*: the CPU gathers writes and sends them in bursts, which is fast, but every read goes all the way to memory, which is slow. Writing into such memory with `memcpy` is fine. Reading from it, or updating it with `+=`, can be ten or a hundred times slower than ordinary memory. Prefer `HOST_CACHED` types for buffers the CPU reads back.

Images

An *image* is an array of *texels* with a format, an extent in up to three dimensions, a number of mip levels and array layers, a sample count, a tiling and usage flags. Images exist because GPUs have hardware for them: texture units that filter and convert formats on the fly, compression, and render targets.

A 1024 × 1024 RGBA image for shader writes and copies, and its memory requirements — code/u1/memory/main.cpp

```
const VkImageCreateInfo info{
    .sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO,
    .imageType = VK_IMAGE_TYPE_2D,
    .format = VK_FORMAT_R8G8B8A8_UNORM,
    .extent = {1024, 1024, 1},
    .mipLevels = 1,
    .arrayLayers = 1,
    .samples = VK_SAMPLE_COUNT_1_BIT,
    .tiling = VK_IMAGE_TILING_OPTIMAL,
    .usage = VK_IMAGE_USAGE_STORAGE_BIT | VK_IMAGE_USAGE_TRANSFER_SRC_BIT,
    .sharingMode = VK_SHARING_MODE_EXCLUSIVE,
    .initialLayout = VK_IMAGE_LAYOUT_UNDEFINED,
};
VkImage image = VK_NULL_HANDLE;
U1_CHECK(vkCreateImage(device.device, &info, nullptr, &image));
VkMemoryRequirements r;
vkGetImageMemoryRequirements(device.device, image, &r);
```

The *format* names the components, their sizes and how to interpret them. `VK_FORMAT_R8G8B8A8_UNORM` has four 8-bit components read as numbers from 0 to 1; `_SRGB` applies the sRGB curve on reads and writes; `_SFLOAT`, `_UINT` and `_SINT` store floating-point, unsigned and signed integer values. Support varies by format, by tiling and by use, and `vkGetPhysicalDeviceFormatProperties` reports it:

Which formats can be storage images, be sampled, or be rendered to — code/u1/memory/main.cpp

```
for (VkFormat format : {VK_FORMAT_R8G8B8A8_UNORM, VK_FORMAT_R8G8B8A8_SRGB,
                        VK_FORMAT_R32_SFLOAT, VK_FORMAT_R16G16B16A16_SFLOAT}) {
    VkFormatProperties properties;
    vkGetPhysicalDeviceFormatProperties(device.physical, format, &properties);
    const VkFormatFeatureFlags f = properties.optimalTilingFeatures;
    std::printf("%-30s storage %-3s sampled %-3s colour attachment %s\n",
                string_VkFormat(format),
                (f & VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT) ? "yes" : "no",
                (f & VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT) ? "yes" : "no",
                (f & VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT) ? "yes" : "no");
}
```

The output shows that the M2 Pro can write `VK_FORMAT_R8G8B8A8_SRGB` from a shader. Many desktop drivers cannot, which is why portable compute code writes `UNORM` and converts to sRGB itself, or writes through a `UNORM` view of an sRGB image created with `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` and `VK_IMAGE_CREATE_EXTENDED_USAGE_BIT`.

The *tiling* decides how texels are arranged in memory. `VK_IMAGE_TILING_LINEAR` stores rows one after another, so the CPU can map and address them, but devices support linear images only for a few formats and uses. `VK_IMAGE_TILING_OPTIMAL` uses whatever arrangement suits the hardware, typically tiles or curves that keep neighbouring texels close in memory, and the arrangement is not documented. Optimal images therefore receive their data through copy commands from buffers, and the copy translates between the two arrangements. On the M2 Pro the optimal 1024 × 1024 image needs exactly its 4 MiB of pixels. On other hardware, expect padding and alignments as large as 64 KiB.

Shaders and render passes reach an image through an *image view*, which selects a range of mip levels and layers and can reinterpret the format. Unit 2 creates views for compute and Unit 4 for rendering.

Finally, every image is always in a *layout*: an arrangement of its data suited to a particular kind of access, such as `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` for copies into it, `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` for sampling, or `VK_IMAGE_LAYOUT_GENERAL`, which allows any access at a possible cost in speed. The course's images start in `VK_IMAGE_LAYOUT_UNDEFINED`, which means their contents may be discarded, and moves between layouts through *layout transitions* that you record in barriers. Chapter 3.2 explains transitions as part of the dependency model, and Chapter 2.4 uses them.

> **COMING FROM OPENGL**
> `glTexStorage2D` allocates a texture and the driver chooses its tiling and layout; `glTexSubImage2D` uploads pixels through a staging copy the driver makes for you. Formats such as `GL_RGBA8` are converted to the GPU's native arrangement behind the scenes, and layout changes never appear in your code. Vulkan makes each of those steps explicit: the staging buffer, the copy, the layout and its transitions.

Staging

Memory the CPU cannot map receives its data in two steps. The program writes into a host-visible *staging buffer*, then records a copy from the staging buffer into the device-local buffer or image, with `vkCmdCopyBuffer` or `vkCmdCopyBufferToImage`. Recording and submitting a copy is the subject of Chapter 1.3, which also measures staging against writing directly into memory that is both device-local and host-visible. On a discrete GPU, staging is the normal path for data the GPU reads repeatedly. On unified memory, and on discrete GPUs with resizable BAR, writing directly is an alternative, and Lab 1.3.3 measures which is faster on your machine.

Reading

- The Vulkan specification, chapters “Memory Allocation” (in particular “Device Memory” and “Host Access to Device Memory Objects”) and “Resource Creation” (buffers, images, image layouts and “Resource Memory Association”), at docs.vulkan.org/spec.
- The Vulkan Guide, “Memory Allocation” and “Formats”.
- The Vulkan Memory Allocator documentation (AMD GPUOpen), the pages “Choosing memory type” and “Recommended usage patterns”, which describe how memory types differ across real hardware.

Labs

LAB 1.2.1 Map your machine’s memory — 45 minutes

Goal. Read your device’s heaps and types and decide what each is for.

Code. `code/u1/memory`

1. Run `build/bin/u1_memory` and save the output.
2. Draw your device’s heaps and memory types in the style of Figure 1.2.1.
3. For each of these, choose a memory type and justify it from the flags: a large buffer the GPU reads every frame and the CPU never touches; a staging buffer for uploads; a buffer the CPU reads results from.

DONE WHEN

- [ ] Your drawing accounts for every heap and type in the output.
- [ ] Each choice names a type index and the flags that justify it, or explains why your device offers no better choice.

EVIDENCE FOR THE NOTEBOOK

- The output, the drawing, the three choices with reasons.

LAB 1.2.2 Round trips and coherence — 1.5 hours

Goal. Handle coherent and non-coherent memory correctly, and let validation check your ranges.

Code. `code/u1/memory`

1. Change the round trip to request `HOST_VISIBLE | HOST_CACHED` and note the type it receives and whether it flushed.
2. Write `invalidateBeforeRead`, the mirror image of `flushHostWrites`, with `vkInvalidateMappedMemoryRanges`.
3. Write a version of each that takes an offset and size inside the allocation and rounds them to `nonCoherentAtomSize`. Call it for bytes 100 to 299 of the buffer, with and without the rounding, and run with validation on. The rules on the range apply whether or not the memory is coherent, so the layer checks them on every device.
4. If your device has a host-visible type without `HOST_COHERENT`, force the round-trip buffer into it and confirm that the flush runs.

DONE WHEN

- [ ] Without rounding, validation reports the misaligned range; with rounding it is silent.
- [ ] The program prints which memory type each buffer received and whether a flush or invalidate ran.

EVIDENCE FOR THE NOTEBOOK

- The validation message for the misaligned range, and your rounding code.

LAB 1.2.3 A thousand buffers in one allocation — 1.5 hours

Goal. Measure what sub-allocation saves.

Code. `code/u1/memory`

1. Run the program and confirm that the thousand buffers do not overlap.
2. Add a variant that gives each of the thousand buffers its own `vkAllocateMemory`, and time both versions of the loop with `std::chrono::steady_clock`, excluding buffer creation.
3. Print `maxMemoryAllocationCount` and `bufferImageGranularity` for your device.
4. Add a `reset()` to `LinearAllocator` that makes the whole block free again, and test that a full block throws and a reset block accepts buffers.

DONE WHEN

- [ ] You have the time per buffer for both strategies on your device.
- [ ] You can say how many separately allocated buffers your device's driver would allow.
- [ ] The overflow and reset tests pass with no validation errors.

EVIDENCE FOR THE NOTEBOOK

- The timings and limits, and the test output.

LAB 1.2.4 Survey formats and image memory — 1 hour

Goal. Learn what your device supports before you rely on it.

Code. `code/u1/memory`

1. Add `VK_FORMAT_R8_UNORM`, `VK_FORMAT_B8G8R8A8_UNORM`, `VK_FORMAT_R32G32B32A32_SFLOAT` and `VK_FORMAT_D32_SFLOAT` to the format survey, and print the linear-tiling features beside the optimal ones.
2. Print the memory requirements of optimal images of 1000 × 1000 and 1024 × 1024 texels, and of a linear 1024 × 1024 image. For the linear one, also print `vkGetImageSubresourceLayout`'s `rowPitch`.

DONE WHEN

- [ ] Your table covers eight formats and both tilings.
- [ ] You can name one format your device can sample but not write from a shader, or explain why it has none.
- [ ] You can explain any difference between the row pitch and 1024 × 4 bytes.

EVIDENCE FOR THE NOTEBOOK

- The table and the image sizes, with one sentence on each surprise.

Problem set

1. A device has heap 0 (8 GiB, device-local) with types `DEVICE_LOCAL` and `DEVICE_LOCAL | HOST_VISIBLE | HOST_COHERENT`, and heap 1 (32 GiB) with types `HOST_VISIBLE | HOST_COHERENT` and `HOST_VISIBLE | HOST_CACHED`. Choose a type for each of: a 2 GiB mesh uploaded once, a 64 KiB block of parameters rewritten every frame, and a 16 MiB buffer of results read by the CPU every frame. Justify each choice.
2. A buffer's requirements are size 1000, alignment 256 and `memoryTypeBits` `0b101`. A linear allocator's next free offset is 4100 in a block of memory type 2. At what offset is the buffer bound, and how much padding does that cost? What happens if the block's memory type is 1?
3. Why is the number of allocations limited, and why does a sub-allocator not suffer from the limit?
4. Explain why writing pixels through a mapped pointer works for a linear image but not for an optimal one. What does `vkCmdCopyBufferToImage` do that the CPU cannot?

5. `nonCoherentAtomSize` is 64. You wrote bytes 100 to 299 of a 4096-byte non-coherent allocation. Which range do you flush, and why may flushing more than you wrote be harmless here but wrong in general?

COMPLETION CHECKLIST

- [ ] I can read a device's heaps and memory types and choose a type for a given use.
- [ ] I can create a buffer and an image, query their requirements, and bind them at valid offsets.
- [ ] I can explain coherence, and flush and invalidate correctly aligned ranges.
- [ ] I can explain tiling, formats and layouts, and query format support.
- [ ] Lab 1.2.1–1.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.3

Commands and submission

TIME	5 hours
YOU BUILD	A program that records fill and copy commands, submits them, waits on a fence and checks the result on the CPU; then measures staging against direct writes and the cost of a submission.
NEEDS	Chapter 1.2.

WHY IT MATTERS

Nothing you create in Vulkan does any work until you record commands and submit them to a queue. Recording and submission are separate so that you can record on any thread, ahead of time, and submit many command buffers in one call, which keeps the driver's cost per command small. The price is that the GPU runs your work later, on its own timeline, and you have to say when the CPU may touch the results. This chapter introduces both timelines and the first two synchronisation tools: barriers inside a command buffer and fences between the GPU and the CPU.

SKIP TEST

1. Name the states of a command buffer, and what moves it between them.
2. Why may you not overwrite a staging buffer immediately after `vkQueueSubmit2` returns, and how do you know when you may?
3. `vkCmdFillBuffer` writes a buffer and `vkCmdCopyBuffer` then reads it. What must be recorded between them, and why does the order of recording not suffice?
4. A program submits one command buffer two hundred times in a single `vkQueueSubmit2`. What must it have declared when it began recording?

If all four are easy, skim the core ideas and do Labs 1.3.3 and 1.3.4.

Core ideas

Command pools and command buffers

A *command buffer* holds a sequence of commands for the device. You allocate command buffers from a *command pool*, which owns their memory and belongs to one queue family: command buffers from a pool may only be submitted to queues of that family.

A pool on the program's queue family, and a primary command buffer from it — code/u1/commands/main.cpp

```
VkCommandPool createPool(const u1::Device& device) {
    const VkCommandPoolCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO,
        .flags = VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT,
        .queueFamilyIndex = device.queueFamily,
    };
    VkCommandPool pool = VK_NULL_HANDLE;
    U1_CHECK(vkCreateCommandPool(device.device, &info, nullptr, &pool));
    return pool;
}

VkCommandBuffer allocate(const u1::Device& device, VkCommandPool pool) {
    const VkCommandBufferAllocateInfo info{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO,
        .commandPool = pool,
        .level = VK_COMMAND_BUFFER_LEVEL_PRIMARY,
        .commandBufferCount = 1,
    };
    VkCommandBuffer cmd = VK_NULL_HANDLE;
    U1_CHECK(vkAllocateCommandBuffers(device.device, &info, &cmd));
    return cmd;
}
```

The flag `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT` allows each command buffer to be reset on its own, either with `vkResetCommandBuffer` or implicitly when recording begins again. Without it, the only way to reuse command buffers is to reset the whole pool with `vkResetCommandPool`, which is cheaper and is what most renderers do once per frame. `VK_COMMAND_POOL_CREATE_TRANSIENT_BIT` tells the driver that command buffers will be short-lived.

A pool and the command buffers allocated from it are *externally synchronised*: two threads must not use them at the same time. Vulkan does not lock anything for you. Programs that record on several threads give each thread its own pool, which Chapter 4.6 does.

Primary command buffers are submitted to queues. *Secondary* command buffers are executed from primaries with `vkCmdExecuteCommands`; they exist so that several threads can record parts of one rendering pass, and Chapter 4.6 uses them too.

Recording

Recording starts with `vkBeginCommandBuffer` and ends with `vkEndCommandBuffer`. Every command in between is a `vkCmd*` call that appends to the buffer and does nothing else. No work starts until submission. Usage flags at the beginning describe how the buffer will be submitted. `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT` promises a single submission, which lets some drivers record more cheaply.

Beginning a command buffer that will be submitted once — code/u1/commands/main.cpp

```
void begin(VkCommandBuffer cmd) {
    const VkCommandBufferBeginInfo info{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
        .flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT,
    };
    U1_CHECK(vkBeginCommandBuffer(cmd, &info));
}
```

A command buffer moves through a small set of states. It starts in the *initial* state. `vkBeginCommandBuffer` moves it to *recording*, and `vkEndCommandBuffer` to *executable*. Submission makes it *pending* until the device finishes with it, after which it is executable again, or *invalid* if it was a one-time submission. Resetting it returns it to initial. A pending command buffer may not be reset, re-recorded or freed, and submitting it again while it is still pending requires `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT`.

Transfer commands

The simplest commands move bytes. They run on any queue whose family supports transfer, which includes every graphics or compute family.

- `vkCmdFillBuffer` writes a repeated 32-bit value; the offset and size must be multiples of 4.
- `vkCmdUpdateBuffer` copies up to 65536 bytes of data that are stored inside the command buffer itself. It suits small constant blocks, and anything larger belongs in a staging buffer.
- `vkCmdCopyBuffer` copies regions between buffers.
- `vkCmdCopyBufferToImage` and `vkCmdCopyImageToBuffer` move texels between buffers and images, translating between a buffer's rows and an image's tiling. `vkCmdCopyImage` copies between images.
- `vkCmdBlitImage`, which scales and converts formats, and `vkCmdClearColorImage` complete the set. Blits require a graphics queue.

Barriers: a first look

Commands recorded in a command buffer are submitted in the order you recorded them, but nothing makes one finish before the next begins. A GPU overlaps commands wherever it can, and writes may sit in caches where a later command cannot see them. When one command reads what another wrote, you must record a *pipeline barrier* between them, which states two things: which earlier work must finish first (its *stage*), and which writes must be made visible to which later accesses (its *access masks*).

A global memory barrier in its synchronization2 form — code/u1/commands/main.cpp

```
void barrier(VkCommandBuffer cmd, VkPipelineStageFlags2 srcStage, VkAccessFlags2 srcAccess,
             VkPipelineStageFlags2 dstStage, VkAccessFlags2 dstAccess) {
    const VkMemoryBarrier2 memory{
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = srcStage,
        .srcAccessMask = srcAccess,
        .dstStageMask = dstStage,
        .dstAccessMask = dstAccess,
    };
    const VkDependencyInfo dependency{
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .memoryBarrierCount = 1,
        .pMemoryBarriers = &memory,
    };
    vkCmdPipelineBarrier2(cmd, &dependency);
}
```

`srcStageMask` and `srcAccessMask` describe the earlier work: here, the stage of the fill command and its writes. `dstStageMask` and `dstAccessMask` describe the later work that must wait and the accesses that must see the writes. Unit 3 is devoted to this model. For now, one barrier between every writer and the reader that follows is enough.

The program below fills a device-local buffer, copies it into a host-visible buffer and reads it on the CPU. It needs two barriers. The first orders the copy's reads after the fill's writes. The second makes the copy's writes available to the host, whose reads are a stage of their own, `VK_PIPELINE_STAGE_2_HOST_BIT`.

Fill, copy to host-visible memory, and wait — code/u1/commands/main.cpp

```
begin(cmd);
vkCmdFillBuffer(cmd, gpu.buffer, 0, bytes, 0xC0FFEE00u);
barrier(cmd, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
        VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_READ_BIT);
const VkBufferCopy region{.srcOffset = 0, .dstOffset = 0, .size = bytes};
vkCmdCopyBuffer(cmd, gpu.buffer, host.buffer, 1, &region);
barrier(cmd, VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
        VK_PIPELINE_STAGE_2_HOST_BIT, VK_ACCESS_2_HOST_READ_BIT);
U1_CHECK(vkEndCommandBuffer(cmd));
u1::submitAndWait(device, cmd);
```

If you remove the first barrier, the program still prints the right answer on the GPU used to write this course, because the fill happened to finish before the copy started. Synchronisation validation reports the hazard all the same.

Synchronisation validation names the hazard, both commands, and the stages and accesses that a barrier must connect

```
$ ./build/bin/u1_commands            # with the barrier after vkCmdFillBuffer removed
[validation error] vkCmdCopyBuffer(): READ_AFTER_WRITE hazard detected. vkCmdCopyBuffer
reads VkBuffer 0x30000000003, which was previously written by vkCmdFillBuffer. No
sufficient synchronization is present to ensure that a read (VK_ACCESS_2_TRANSFER_READ_BIT)
at VK_PIPELINE_STAGE_2_COPY_BIT does not conflict with a prior write
(VK_ACCESS_2_TRANSFER_WRITE_BIT) at VK_PIPELINE_STAGE_2_CLEAR_BIT.
Buffer copy region 0: {
  offset = 0,
  size = 1048576
}
fill, copy and read back 1 MiB: ok
...
validation reported 1 error(s)
```

PITFALL

A program that gives the right result without a required barrier is still wrong, because its result depends on timing. The same program may fail on another GPU or driver, or on the same GPU when the copy is larger or the queue is busier. Synchronisation validation checks the rules instead of the outcome, which makes it the only reliable test.

Submission and fences

`vkQueueSubmit2` hands batches of command buffers to a queue. Each `VkSubmitInfo2` is one batch: its command buffers, and optionally semaphores to wait for before it starts and to signal when it completes (Chapter 3.4). The call returns as soon as the work is queued, usually long before the device has started it.

To learn when the work has finished, pass a *fence*. A fence is a two-state object that the queue signals when every batch in the submission has completed. The host waits for it with `vkWaitForFences`, which takes a timeout in nanoseconds, or polls it with `vkGetFenceStatus`. Before a fence can be used again it must be reset with `vkResetFences`. Unit 1's `submitAndWait` creates a fence, submits one command buffer, waits and destroys the fence.

Submitting one command buffer and waiting for it — code/u1/shared/basics.hpp

```
inline void submitAndWait(const Device& device, VkCommandBuffer cmd) {
    const VkFenceCreateInfo fenceInfo{.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO};
    VkFence fence = VK_NULL_HANDLE;
    U1_CHECK(vkCreateFence(device.device, &fenceInfo, nullptr, &fence));

    const VkCommandBufferSubmitInfo cmdInfo{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO,
        .commandBuffer = cmd,
    };
    const VkSubmitInfo2 submit{
        .sType = VK_STRUCTURE_TYPE_SUBMIT_INFO_2,
        .commandBufferInfoCount = 1,
        .pCommandBufferInfos = &cmdInfo,
    };
    U1_CHECK(vkQueueSubmit2(device.queue, 1, &submit, fence));
    U1_CHECK(vkWaitForFences(device.device, 1, &fence, VK_TRUE, UINT64_MAX));
    vkDestroyFence(device.device, fence, nullptr);
}
```

Creating and destroying a fence for every submission is wasteful, and Lab 1.3.4 measures by how much. Waiting immediately after every submission costs more, because the CPU sits idle while the GPU works, and then the GPU sits idle while the CPU records the next batch. Real programs keep several submissions in flight, as Chapter 3.6 shows.

Two timelines

Figure 1.3.1 shows what happens in time. The host records and submits, the device runs the commands later at its own pace, and the fence connects the two when the host needs a result. Between submission and the wait the host is free to do other work, but it must not touch anything the submitted commands use. In

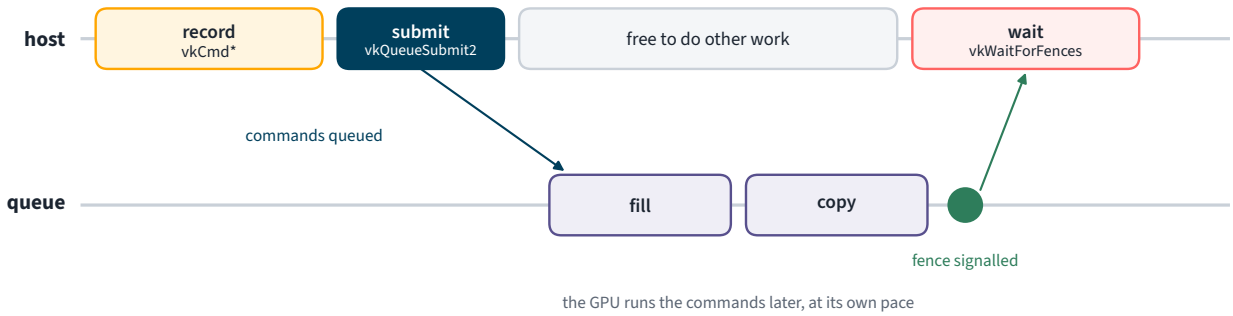


Figure 1.3.1. The host and device timelines. Submission returns immediately, the device runs the work later, and the fence tells the host when it has finished.

particular it must not overwrite a staging buffer whose copy has not yet run, free memory the commands read, or reset the command buffer while it is pending.

> **KEY IDEA**
> Every Vulkan program runs on at least two timelines: the host's and each queue's. They meet only through submission and synchronisation objects such as fences, and everything you share between them must be handed over at one of those places.

Staging in practice

The program's second experiment uploads 64 MiB into a device-local buffer through a staging buffer, as Chapter 1.2 described. The barrier at the end makes the copy visible to the compute shaders that will read the buffer.

Uploading through a staging buffer, timed from the first byte written to the copy's completion — code/u1/commands/main.cpp

```
const auto start = std::chrono::steady_clock::now();
std::memcpy(staging.mapped, source.data(), bytes);
begin(cmd);
const VkBufferCopy region{.size = bytes};
vkCmdCopyBuffer(cmd, staging.buffer, target.buffer, 1, &region);
barrier(cmd, VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
        VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_ACCESS_2_SHADER_STORAGE_READ_BIT);
U1_CHECK(vkEndCommandBuffer(cmd));
u1::submitAndWait(device, cmd);
const double staged = millisecondsSince(start);
```

The program then writes the same 64 MiB directly into memory that is both device-local and host-visible, when the device has such a type, and compares the two. It finishes by measuring what a submission costs, waiting for each submission in turn and then batching two hundred submissions of the same empty command buffer into one call.

Two hundred synchronous submissions, then the same two hundred in one batch code/u1/commands/main.cpp

```
auto start = std::chrono::steady_clock::now();
for (int i = 0; i < count; ++i) u1::submitAndWait(device, cmd);
const double oneByOne = millisecondsSince(start) / count;

std::vector<VkCommandBufferSubmitInfo> infos(
    count, {.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO, .commandBuffer = cmd});
const VkSubmitInfo2 batch{
    .sType = VK_STRUCTURE_TYPE_SUBMIT_INFO_2,
    .commandBufferInfoCount = static_cast<uint32_t>(infos.size()),
    .pCommandBufferInfos = infos.data(),
};
start = std::chrono::steady_clock::now();
U1_CHECK(vkQueueSubmit2(device.queue, 1, &batch, VK_NULL_HANDLE));
U1_CHECK(vkQueueWaitIdle(device.queue));
const double batched = millisecondsSince(start) / count;
```

u1_commands with validation off on an Apple M2 Pro

```
$ VKF_VALIDATION=0 ./build/bin/u1_commands
fill, copy and read back 1 MiB: ok
64 MiB through a staging buffer: 5.63 ms (11.9 GB/s)
64 MiB written straight into device-local, host-visible memory: 3.94 ms (17.0 GB/s)
empty command buffer: 19.4 us per submit and wait, 0.43 us each when 200 are batched
```

On the M2 Pro, with unified memory, writing directly is faster than staging, because staging writes the data twice. On a discrete GPU the answer depends on the bus and on how the data is used. Writing through a resizable BAR mapping sends the CPU's writes across PCIe as they happen, which avoids the second copy but keeps the CPU busy for the whole transfer. A staged copy lets the CPU write at the speed of system memory and leaves the transfer to the GPU's copy engine, which can overlap other work. Reading BAR memory from the CPU is slow either way. Measure on your own machine in Lab 1.3.3.

The submission figures show why batching matters. A submission followed by a wait costs about 20 µs, most of it in the round trip between the CPU and the GPU. Batched, each command buffer costs well under a microsecond, so the cost per command buffer stays small when you submit many at once and do not wait.

PITFALL

The batch above lists the same command buffer two hundred times. The first copy is pending while the second is submitted, so the command buffer must have been recorded with `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT`. Without it, the validation layer reports that the command buffer "is already in use and is not marked for simultaneous use". The flag can make recording or execution slower on some drivers, so use it only when you need it.

Declaring that a command buffer may be pending more than once code/u1/commands/main.cpp

```
const VkCommandBufferBeginInfo info{
    .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
    .flags = VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT,
};
```

COMING FROM OPENGL

OpenGL records commands into a driver-managed buffer as you call them, and decides itself when to send them to the GPU. `glFlush` asks it to send what it has, and `glFinish` waits for everything to complete, like a `vkQueueWaitIdle`. A fence sync object, created with `glFenceSync` and waited on with `glClientWaitSync`, corresponds to a Vulkan fence. Between commands, the OpenGL driver tracks which buffer and texture each command writes and inserts the barriers itself. The exception is shader writes to buffers and images, for which OpenGL 4.2 introduced `glMemoryBarrier` because the driver cannot see which addresses a shader will touch. Vulkan asks you for every barrier, including those for copies.

Reading

- The Vulkan specification, chapters “Command Buffers” (lifecycle, recording and submission), “Copy Commands”, “Clear Commands”, and the “Fences” section of “Synchronization and Cache Control”.
- The Vulkan Guide, “Queues”, “Synchronization” and “Common Pitfalls for New Vulkan Developers”.
- Khronos, “Synchronization Examples” in the Vulkan Guide, the transfer cases only; Unit 3 covers the rest.

Labs

LAB 1.3.1 Fill, update, copy and read back — 1 hour

Goal. Record and verify a short chain of transfer commands.

Code. `code/u1/commands`

1. Run `build/bin/u1_commands` and confirm `fill, copy and read back 1 MiB: ok`.
2. After the fill, add a `vkCmdUpdateBuffer` that writes the first 256 bytes with the values 0 to 63 as `uint32_t`, with the barrier it needs.
3. Extend the check on the CPU to expect both patterns.

DONE WHEN

- [ ] The read-back data matches both patterns.
- [ ] The program runs with no validation errors, and you can say which stages and accesses your new barrier connects.

EVIDENCE FOR THE NOTEBOOK

- The recorded commands, with a sentence per barrier.

LAB 1.3.2 Remove each barrier in turn — 45 minutes

Goal. See what synchronisation validation reports, and compare it with what happens.

Code. `code/u1/commands`

1. Remove the barrier between the fill and the copy, run, and record the message and the printed result.
2. Restore it and remove the barrier between the copy and the host read instead. Run again and record what happens.
3. Restore both.

DONE WHEN

- [ ] For each removal you have recorded whether validation reported it, what it named, and whether the result was still correct.
- [ ] You can explain why a correct result proves nothing here.

EVIDENCE FOR THE NOTEBOOK

- Both messages, or the absence of one, with your explanation.

LAB 1.3.3 Staging against direct writes — 1.5 hours

Goal. Find out which upload path is faster on your machine, and when.

Code. `code/u1/commands`

1. Run the program with `VKF_VALIDATION=0` three times and record both upload timings.
2. Make the size a command-line option and measure 1, 4, 16, 64 and 256 MiB.

3. On a discrete GPU, find out whether resizable BAR is enabled: is there a device-local, host-visible type, and is its heap the size of VRAM?

DONE WHEN

- [ ] You have a table of both paths at five sizes, from three runs each.
- [ ] You can explain the result from your device's memory types.

EVIDENCE FOR THE NOTEBOOK

- The table and the memory types it depends on.

LAB 1.3.4 What a submission costs — 1.5 hours

Goal. Measure submission overhead and the ways to reduce it.

Code. `code/u1/commands`

1. Remove `VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT`, run with validation on, and record the message.
2. Restore it. Change `submitAndWait` to create its fence once and reset it before each submission, and measure the change in cost per synchronous submission.
3. Measure the batched cost per command buffer for batches of 1, 10, 100 and 1000.

DONE WHEN

- [ ] You have the cost per synchronous submission with and without fence reuse, and the batched cost at four batch sizes, with validation off.
- [ ] The program runs with no validation errors.

EVIDENCE FOR THE NOTEBOOK

- The timings, and a sentence on what dominates the cost of a synchronous submission.

Problem set

1. Draw the lifecycle of a command buffer allocated from a pool created with `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT`, from allocation through two submissions to freeing. Mark the transitions that are invalid while it is pending.
2. Why are command pools externally synchronised instead of being made thread-safe by the driver, and how does a program that records on eight threads organise its pools?
3. A program calls `vkQueueSubmit2` for an upload and immediately writes the next upload's data into the same staging buffer. What can go wrong, and what are two correct alternatives?
4. When would you use `vkCmdUpdateBuffer` and when a staging buffer with `vkCmdCopyBuffer`? State the restrictions on the first.
5. A synchronous submission costs about 20 µs on the machine above. Why is "a 60 Hz frame can afford 800 of them" the wrong conclusion?

COMPLETION CHECKLIST

- [ ] I can create command pools and command buffers, record transfer commands, and explain the command buffer lifecycle.
- [ ] I can record a barrier between a writer and a reader, and between the device and the host.
- [ ] I can submit work with `vkQueueSubmit2` and wait for it with a fence.
- [ ] I can explain the host and device timelines and what each may touch when.
- [ ] Lab 1.3.1–1.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.4

Shaders, SPIR-V and pipelines

TIME	5 hours
YOU BUILD	A compute shader compiled offline to SPIR-V, validated and disassembled; a pipeline layout with a descriptor set and push constants; three pipelines that differ only in a specialisation constant; and a pipeline cache saved to disk and loaded again.
NEEDS	Chapter 1.3. The SDK's `glslc`, `spirv-val` and `spirv-dis`.

WHY IT MATTERS

An OpenGL driver compiled your GLSL at run time, with its own compiler, and sometimes compiled it again when unrelated state changed. A Vulkan driver never sees GLSL. It receives SPIR-V, a binary intermediate language you produced at build time, together with a complete description of everything the shader will touch, and it compiles the result once, when you create a pipeline. To keep a Vulkan program free of the stalls that come from compiling shaders in the middle of a frame, you need to know what goes into a pipeline, what can vary afterwards, and when compilation happens.

SKIP TEST

1. What is SPIR-V, and why does Vulkan consume it instead of GLSL?
2. What does a pipeline layout describe, and why must it exist before the pipeline does?
3. How does a specialisation constant differ from a push constant, and when would you use each?
4. What does a pipeline cache store, and when does loading one from disk not help?

If all four are easy, skim the core ideas and do Labs 1.4.2 and 1.4.4.

Core ideas

A compute shader in GLSL

This unit's shaders are written in GLSL 4.60 with the Vulkan rules that the `GL_KHR_vulkan_glsl` extension defines. The differences from OpenGL's GLSL are about how a shader reaches the outside world. Resources are grouped into *descriptor sets* and named by `set` and `binding` numbers instead of by name. Small parameters arrive in a *push constant* block. Constants that are fixed when the pipeline is created carry a `constant_id`. A `uniform` of a plain type such as `float` must be inside a block.

The shader for this chapter writes a pseudo-random pattern into a buffer. Its interface declares all three kinds of input:

The shader's interface: a specialisable workgroup size, two push constants and one storage buffer — code/u1/pipeline/fill.comp

```
layout(local_size_x_id = 0) in;

layout(push_constant) uniform Push {
    uint seed;
    uint count;
} push;

layout(std430, set = 0, binding = 0) writeonly buffer Output {
    uint values[];
};
```

`local_size_x_id = 0` makes the workgroup's width a specialisation constant with ID 0, set when the pipeline is created. Unit 2 explains workgroups in depth. The push constant block holds a seed and an element count. The storage buffer at set 0, binding 0 holds the output; `std430` fixes the layout rules for its members, and `writeonly` promises that the shader never reads it.

One invocation per element — code/u1/pipeline/fill.comp

```
void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i ≥ push.count) return;
    values[i] = push.seed ^ (i * 2654435761u);
}
```

Each invocation handles one element, chosen by its global index. The dispatch covers whole workgroups, so when the count is not a multiple of the workgroup size the last group has invocations past the end of the data. The early return keeps them from writing outside the buffer.

SPIR-V

SPIR-V is a binary intermediate language for shaders and compute kernels, defined by Khronos. A module is a stream of 32-bit words that starts with the magic number `0x07230203`, followed by declarations of capabilities, types, constants and variables, decorations that connect them to the outside world, and functions in static single-assignment form. Vulkan 1.3 accepts SPIR-V up to version 1.6.

Taking SPIR-V instead of source text removes a compiler front end from every driver. Each OpenGL driver had its own GLSL parser, with its own bugs and its own interpretation of corner cases, and every program paid to parse its shaders at start-up. With SPIR-V, parsing and most optimisation happen once, at build time, and any language that compiles to SPIR-V can be used, including GLSL, HLSL and Slang.

The SDK provides the tools. `glslc`, the command-line compiler from Google's shaderc project, compiles GLSL with Khronos's reference compiler, glslang. `spirv-val` checks a module against the SPIR-V specification and Vulkan's rules for it, `spirv-dis` turns it into readable text, and `spirv-opt` optimises it. The course's build runs `glslc` for every shader through the CMake function `vkf_shaders`.

The fill shader compiled, validated and disassembled, with some declarations elided

```
$ glslc --target-env=vulkan1.3 -O fill.comp -o fill.spv
$ spirv-val --target-env vulkan1.3 fill.spv
$ spirv-dis fill.spv
; SPIR-V
; Version: 1.6
; Generator: Google Shaderc over Glslang; 11
; Bound: 50
; Schema: 0
               OpCapability Shader
          %1 = OpExtInstImport "GLSL.std.450"
               OpMemoryModel Logical GLSL450
               OpEntryPoint GLCompute %4 "main" %gl_GlobalInvocationID %21 %35
               OpExecutionModeId %4 LocalSizeId %7 %uint_1 %uint_1
               OpDecorate %7 SpecId 0
               OpDecorate %gl_GlobalInvocationID BuiltIn GlobalInvocationId
               ...
               OpDecorate %35 Binding 0
```

```
        OpDecorate %35 DescriptorSet 0
        ...
 %4 = OpFunction %void None %3
        ...
%16 = OpAccessChain %_ptr_Input_uint %gl_GlobalInvocationID %uint_0
%17 = OpLoad %uint %16
%25 = OpAccessChain %_ptr_PushConstant_uint %21 %int_1
%26 = OpLoad %uint %25
%28 = OpUGreaterThanEqual %bool %17 %26
      OpSelectionMerge %30 None
      OpBranchConditional %28 %29 %30
%29 = OpLabel
      OpBranch %48
%30 = OpLabel
%38 = OpAccessChain %_ptr_PushConstant_uint %21 %int_0
%39 = OpLoad %uint %38
%42 = OpIMul %uint %17 %uint_2654435761
%43 = OpBitwiseXor %uint %39 %42
%45 = OpAccessChain %_ptr_StorageBuffer_uint %35 %int_0 %17
      OpStore %45 %43
      ...
```

The disassembly shows the interface explicitly. `OpEntryPoint` names the function `main` and lists the variables it uses from outside. `OpExecutionModeId ... LocalSizeId %7 ...` gives the workgroup size, whose width is `%7`, a specialisation constant decorated `SpecId 0`. The storage buffer is a variable decorated `DescriptorSet 0` and `Binding 0`. The body loads `gl_GlobalInvocationID.x`, compares it with the second push constant, multiplies, applies the exclusive-or and stores the result through a pointer into the buffer.

The driver translates SPIR-V into the GPU's own instruction set when the program creates a pipeline (Figure 1.4.1). On macOS there is a further step: MoltenVK translates the SPIR-V into the Metal Shading Language with the SPIRV-Cross library, and Metal compiles that.

> **PITFALL**
> Because MoltenVK goes through Metal's shading language, a name in your shader that collides with a name in Metal's standard library can break pipeline creation on macOS while working everywhere else. A specialisation constant called `bias` did this while the course was being written. If a pipeline fails only under MoltenVK, look for such a collision and rename.

Shader modules

A *shader module* wraps SPIR-V code for the device. Creating one is cheap, because the driver keeps the words and compiles them only when a pipeline is created. The code is passed as a size in bytes and a pointer to 32-bit words.



Figure 1.4.1. From GLSL to a pipeline. Compilation to SPIR-V happens at build time. The driver compiles SPIR-V to the GPU's own instructions when the pipeline is created, with the layout and the specialisation constants known.

Reading a SPIR-V file and creating a shader module — code/u1/shared/basics.hpp

```
inline std::vector<uint32_t> readSpirv(const std::string& path) {
    std::ifstream file(path, std::ios::binary | std::ios::ate);
    if (!file) throw std::runtime_error("cannot open " + path);
    const auto bytes = static_cast<size_t>(file.tellg());
    if (bytes == 0 || bytes % 4 != 0) throw std::runtime_error(path + " is not SPIR-V");
    std::vector<uint32_t> words(bytes / 4);
    file.seekg(0);
    file.read(reinterpret_cast<char*>(words.data()), static_cast<std::streamsize>(bytes));
    return words;
}

inline VkShaderModule createShaderModule(VkDevice device, const std::vector<uint32_t>& code) {
    const VkShaderModuleCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO,
        .codeSize = code.size() * sizeof(uint32_t),
        .pCode = code.data(),
    };
    VkShaderModule module = VK_NULL_HANDLE;
    U1_CHECK(vkCreateShaderModule(device, &info, nullptr, &module));
    return module;
}
```

The build tells the program where its compiled shaders are — code/u1/pipeline/main.cpp

```
const std::vector<uint32_t> spirv =
    u1::readSpirv(std::string(VKF_SHADER_DIR) + "/fill.comp.spv");
VkShaderModule module = u1::createShaderModule(device.device, spirv);
```

The build compiles with `-O`, which optimises, and `-g`, which keeps names and source lines in the module so that validation messages and capture tools can refer to them. The debug information more than doubles the module: 506 words against 252 without it. It costs nothing at run time, because the driver discards it.

Pipeline layouts

A shader's interface must be described to the driver before the shader can be compiled into a pipeline, so that the driver knows where each resource will be found. The *pipeline layout* is that description: an array of *descriptor set layouts*, one for each set index the shaders use, and the ranges of push constants they read.

A descriptor set layout with one storage buffer, and a pipeline layout with eight bytes of push constants — code/u1/pipeline/main.cpp

```
const VkDescriptorSetLayoutBinding binding{
    .binding = 0,
    .descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
    .descriptorCount = 1,
    .stageFlags = VK_SHADER_STAGE_COMPUTE_BIT,
};
const VkDescriptorSetLayoutCreateInfo setInfo{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
    .bindingCount = 1,
    .pBindings = &binding,
};
VkDescriptorSetLayout setLayout = VK_NULL_HANDLE;
U1_CHECK(vkCreateDescriptorSetLayout(device.device, &setInfo, nullptr, &setLayout));

const VkPushConstantRange push{
    .stageFlags = VK_SHADER_STAGE_COMPUTE_BIT, .offset = 0, .size = 8};
const VkPipelineLayoutCreateInfo layoutInfo{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO,
    .setLayoutCount = 1,
    .pSetLayouts = &setLayout,
    .pushConstantRangeCount = 1,
    .pPushConstantRanges = &push,
};
VkPipelineLayout layout = VK_NULL_HANDLE;
U1_CHECK(vkCreatePipelineLayout(device.device, &layoutInfo, nullptr, &layout));
```

The descriptor set layout says that binding 0 is one storage buffer, visible to compute shaders. It describes no particular buffer. Chapter 1.5 allocates sets that follow this layout and points them at real buffers.

Push constants are a small block of bytes, at least 128 and on some devices several kilobytes (`maxPushConstantsSize`), that you write into the command buffer itself with `vkCmdPushConstants`. They need no buffer and no descriptor, which makes them the cheapest way to pass per-dispatch parameters such as counts, offsets or a seed. A push constant block uses the `std430` layout rules, and the range in the pipeline layout must cover every byte the shaders read.

Compute pipelines

A compute pipeline combines one shader stage with a pipeline layout. Creating it is the expensive step, because the driver compiles the SPIR-V for its hardware, using everything it knows from the layout and the specialisation constants.

A compute pipeline with its workgroup width supplied as a specialisation constant — code/u1/pipeline/main.cpp

```
const VkSpecializationMapEntry entry{
    .constantID = 0, .offset = 0, .size = sizeof(uint32_t)};
const VkSpecializationInfo specialization{
    .mapEntryCount = 1,
    .pMapEntries = &entry,
    .dataSize = sizeof(localSize),
    .pData = &localSize,
};
const VkComputePipelineCreateInfo info{
    .sType = VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO,
    .stage =
        {
            .sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
            .stage = VK_SHADER_STAGE_COMPUTE_BIT,
            .module = module,
            .pName = "main",
            .pSpecializationInfo = &specialization,
        },
    .layout = layout,
};
VkPipeline pipeline = VK_NULL_HANDLE;
const auto start = std::chrono::steady_clock::now();
U1_CHECK(vkCreateComputePipelines(device.device, cache, 1, &info, nullptr, &pipeline));
```

A *specialisation constant* is a constant whose value is supplied at pipeline creation. `VkSpecializationInfo` maps constant IDs to bytes in a data block; the driver substitutes the values before it optimises, so a loop bound or a workgroup size behaves like a literal in the source. The program builds three pipelines from one shader module, with workgroups of 64, 128 and 256 invocations. A push constant, by contrast, can change between dispatches but is an ordinary run-time value to the compiler. Use specialisation for values that change rarely and matter to the optimiser, and push constants for values that change often.

> **PITFALL**
> When `glslc` targets Vulkan 1.3 it expresses a specialised workgroup size with the `LocalSizeId` execution mode, which requires the `maintenance4` feature. Without the feature, validation reports that “LocalSizeId is used but maintenance4 feature was not enabled”. This is why Chapter 1.1′s device enables `maintenance4`. Targeting Vulkan 1.2 produces the older form instead, a `WorkgroupSize` built-in made of specialisation constants, which SPIR-V 1.6 deprecates.

Creating a pipeline can take anything from a fraction of a millisecond to hundreds of milliseconds, depending on the shader and the driver. Create pipelines when the program starts or a level loads, never in the middle of a frame. Vulkan has no way to change a pipeline afterwards, so nothing will make the driver recompile it later.

> **KEY IDEA**
> A Vulkan pipeline is compiled once, from SPIR-V, with everything it depends on known: its layout, its specialisation constants and, for graphics, its fixed-function state. It does not change afterwards, so no dispatch or draw triggers a hidden recompile.

Pipeline caches

A *pipeline cache* lets the driver keep what it compiled. You pass a `VkPipelineCache` to `vkCreateComputePipelines`; the driver looks up each pipeline's description in it and stores new results in it. `vkGetPipelineCacheData` returns the contents as bytes, which a program saves to disk and passes back as initial data on its next run.

Creating a cache, empty or from saved data — code/u1/pipeline/main.cpp

```
const VkPipelineCacheCreateInfo cacheInfo{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_CACHE_CREATE_INFO,
    .initialDataSize = initial.size(),
    .pInitialData = initial.data(),
};
VkPipelineCache cache = VK_NULL_HANDLE;
U1_CHECK(vkCreatePipelineCache(device.device, &cacheInfo, nullptr, &cache));
```

Saving the cache when the program finishes — code/u1/pipeline/main.cpp

```
size_t size = 0;
U1_CHECK(vkGetPipelineCacheData(device.device, cache, &size, nullptr));
std::vector<char> data(size);
U1_CHECK(vkGetPipelineCacheData(device.device, cache, &size, data.data()));
std::ofstream(cachePath, std::ios::binary)
    .write(data.data(), static_cast<std::streamsize>(size));
```

The data begins with a header that names the vendor, the device and a `pipelineCacheUUID` that changes whenever the driver's compiler does. The specification requires a driver to ignore data from another device or driver version. A file can also be truncated or corrupted, however, so the program checks the header itself before trusting the file. Production programs also store their own size and checksum beside the data.

Accepting saved data only for this vendor, device and driver — code/u1/pipeline/main.cpp

```
bool cacheMatches(const std::vector<char>& data,
                  const VkPhysicalDeviceProperties& properties) {
    if (data.size() < sizeof(VkPipelineCacheHeaderVersionOne)) return false;
    VkPipelineCacheHeaderVersionOne header;
    std::memcpy(&header, data.data(), sizeof(header));
    return header.headerVersion == VK_PIPELINE_CACHE_HEADER_VERSION_ONE &&
           header.vendorID == properties.vendorID && header.deviceID == properties.deviceID &&
           std::memcmp(header.pipelineCacheUUID, properties.pipelineCacheUUID, VK_UUID_SIZE) ==
               0;
}
```

Two runs of `u1_pipeline` on an Apple M2 Pro

```
$ ./build/bin/u1_pipeline     # first run
fill.comp.spv: 506 words, magic number 0x07230203
pipeline cache: cold (0 bytes loaded from u1_pipeline.cache)
local size  64: pipeline created in 1.15 ms
local size 128: pipeline created in 0.34 ms
local size 256: pipeline created in 0.28 ms
saved 2660 bytes of pipeline cache to u1_pipeline.cache
$ ./build/bin/u1_pipeline     # second run
fill.comp.spv: 506 words, magic number 0x07230203
pipeline cache: warm (2660 bytes loaded from u1_pipeline.cache)
local size  64: pipeline created in 0.93 ms
local size 128: pipeline created in 0.31 ms
local size 256: pipeline created in 0.32 ms
saved 2660 bytes of pipeline cache to u1_pipeline.cache
```

The timings mostly reflect the M2 Pro's driver. Metal keeps its own cache of compiled shaders, shared by every program on the machine, so even the first run with an empty Vulkan cache found the compiled code, and each pipeline took between 0.28 and 1.15 ms. The first creation of this pipeline on the machine took 81 ms. Desktop drivers keep caches of their own on disk too, so a "cold" measurement is cold only the first time. Chapter 4.6 shows how to measure despite that.

> **COMING FROM OPENGL**
> OpenGL compiles GLSL source at run time with `glShaderSource`, `glCompileShader` and `glLinkProgram`, and you find each uniform by name with `glGetUniformLocation`. A program object holds no blending, depth or vertex-format state, so drivers sometimes compile a shader again when such state changes, which shows up as a stall at an unpredictable draw call. `glGetProgramBinary` is the counterpart of a pipeline cache. OpenGL 4.6 can also accept SPIR-V through `glShaderBinary` and `glSpecializeShader`, but macOS stops at OpenGL 4.1.

Reading

- The Vulkan specification, chapters "Shaders" and "Pipelines" (compute pipelines, specialisation constants, pipeline cache), and the appendix "Vulkan Environment for SPIR-V".
- The SPIR-V specification, section 2, in particular "Physical Layout of a SPIR-V Module and Instruction", at registry.khronos.org/SPIR-V.
- The `GL_KHR_vulkan_glsl` extension specification, which lists every difference between GLSL for OpenGL and for Vulkan.
- The Vulkan Guide, "What is SPIR-V", "Push Constants" and "Pipeline Cache".

Labs

LAB 1.4.1 Compile, validate, disassemble — 1 hour

Goal. Read SPIR-V well enough to find your own code in it.

Code. `code/u1/pipeline`

1. Compile `fill.comp` with `glslc --target-env=vulkan1.3` four ways: with and without `-O`, and with and without `-g`. Record the size of each module in words.
2. Validate each with `spirv-val --target-env vulkan1.3`.
3. Disassemble the optimised module without debug information and annotate it: the entry point and its interface, the execution mode, every decoration, and the instructions that implement each line of `main`.

DONE WHEN

- [ ] You have four sizes and four clean validations.
- [ ] Every line of `main` is matched to its instructions in your annotation.

EVIDENCE FOR THE NOTEBOOK

- The annotated disassembly.

LAB 1.4.2 Break the pipeline on purpose — 1 hour

Goal. Recognise the messages that bad shader interfaces produce.

Code. `code/u1/pipeline`

1. In a copy of `basics.hpp`, stop enabling `maintenance4` and run `u1_pipeline`. Record the message and the call that produced it.

2. Restore it. Set `pName` to `"main2"` and run. Record both messages, the validation layer's and the driver's.
3. Restore it. Compile `fill.comp` with `--target-env=vulkan1.2`, disassemble it, and find how the workgroup size is expressed now.

DONE WHEN

- [ ] You have recorded both validation messages with the VUID each cites.
- [ ] You can explain the difference between the two SPIR-V forms of a specialised workgroup size.

EVIDENCE FOR THE NOTEBOOK

- The messages and the two disassembly excerpts.

LAB 1.4.3 Two specialisation constants

1.5 hours

Goal. Pass several constants through one specialisation block.

Code. `code/u1/pipeline`

1. Add a second specialisation constant to `fill.comp`, `layout(constant_id = 1) const uint stride = 1;`, and use it to scale the index the shader writes to.
2. Pass both constants from a C++ struct, with one `VkSpecializationMapEntry` per member and the right offsets.
3. Disassemble the result and find both `SpecId` decorations.
4. Create pipelines for all combinations of three workgroup sizes and two strides.

DONE WHEN

- [ ] Six pipelines are created with no validation errors.
- [ ] Your map entries use `offsetof` instead of hand-written offsets.

EVIDENCE FOR THE NOTEBOOK

- The shader, the struct and its map entries.

LAB 1.4.4 Warm and cold caches

1.5 hours

Goal. Measure what a pipeline cache saves on your machine, and when.

Code. `code/u1/pipeline`

1. Run `u1_pipeline` twice and record the creation times and the cache size.
2. Change one byte of the saved file's UUID and confirm that the program treats the cache as cold.
3. Extend the program to create twenty pipelines, with workgroup widths of 32 to 640 in steps of 32, and time the whole set with a cache, without one, and with a cache loaded from disk.

DONE WHEN

- [ ] You have the three timings for the set of twenty pipelines, from three runs each.
- [ ] You can explain your results in terms of your driver's own caching, with evidence such as a first-ever run on a new shader.

EVIDENCE FOR THE NOTEBOOK

- The timings and your explanation.

Problem set

1. Give two reasons why Vulkan consumes SPIR-V instead of GLSL, and one thing a program gives up by not shipping its shader source.
2. A shader's push constant block contains a `uint` followed by a `vec4`. Give each member's offset and the size of the `VkPushConstantRange` the pipeline layout needs.
3. For each of these, choose a specialisation constant or a push constant and explain why: the side of a tile in shared memory; the number of elements in this dispatch; a flag that enables an expensive debug path.
4. A program copies its pipeline cache file from a development machine with one GPU to a user's machine with another. What happens when the user's machine loads it, and what would you do instead?
5. What happens to the elements beyond `count` if the bounds check in `main` is removed and `count` is not a multiple of the workgroup width? Which tool would tell you, and when?

COMPLETION CHECKLIST

- [ ] I can compile GLSL to SPIR-V, validate it, and read its disassembly.
- [ ] I can create shader modules, descriptor set layouts, pipeline layouts and compute pipelines.
- [ ] I can choose between specialisation constants and push constants, and supply both.
- [ ] I can save and reload a pipeline cache and validate its header.
- [ ] Lab 1.4.1–1.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.5

Descriptors and your first dispatch

TIME	6 hours
YOU BUILD	SAXPY, `y = a·x + y`, as a complete Vulkan program written against the raw API: buffers, a descriptor set, a pipeline, ten dispatches with the barriers between them, and a check of every element on the CPU. Then vector addition, and descriptor sets reused across dispatches.
NEEDS	Chapters 1.2 to 1.4.

WHY IT MATTERS

Everything in Unit 1 so far has been preparation, and this chapter connects the pieces. The shader from Chapter 1.4 reaches real buffers through a *descriptor set*, the pipeline is bound into a command buffer, and the GPU computes a result. SAXPY is the "hello, world" of GPU computing because it has every part of a real compute program and nothing else, so each line of it teaches something you will use in every program that follows.

SKIP TEST

1. What is a descriptor, and which four objects does a program create to give a shader access to one buffer?
2. How many workgroups does a dispatch need for 1,000,003 elements with 256 invocations per workgroup, and what must the shader do about the extra invocations?
3. A dispatch reads and writes a buffer that the previous dispatch wrote. Write the barrier between them.
4. Give the `std430` offset of each member of `struct { vec3 p; float m; vec2 v; }`.

If all four are easy, read the section on `std430` and do Labs 1.5.3 and 1.5.4.

Core ideas

Descriptors

A shader refers to its resources by set and binding numbers. A *descriptor* is the small record that connects such a binding to an actual resource: for a buffer, its address and size; for an image, its view, layout and format. Descriptors are grouped into *descriptor sets* that follow a *descriptor set layout*, and sets are allocated from a *descriptor pool*. Figure 1.5.1 shows how the pieces fit for SAXPY.

Each binding has a *descriptor type* that says what kind of resource it holds and how the shader may use it. Compute programs mostly use these:

- `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER`: a buffer the shader may read and write, of any size up to `maxStorageBufferRange`.
- `VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER`: a read-only buffer of at most `maxUniformBufferRange` bytes (at least 16 KiB, and 64 KiB on many devices), which some hardware reads through a faster path.
- `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`: an image the shader reads and writes texel by texel (Chapter 2.4).
- `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, `VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE` and `VK_DESCRIPTOR_TYPE_SAMPLER`: images read through the texture hardware with filtering (Chapters 2.4 and 4.3).

The SAXPY shader declares two storage buffers, `x` read-only and `y` read-write, and two push constants:

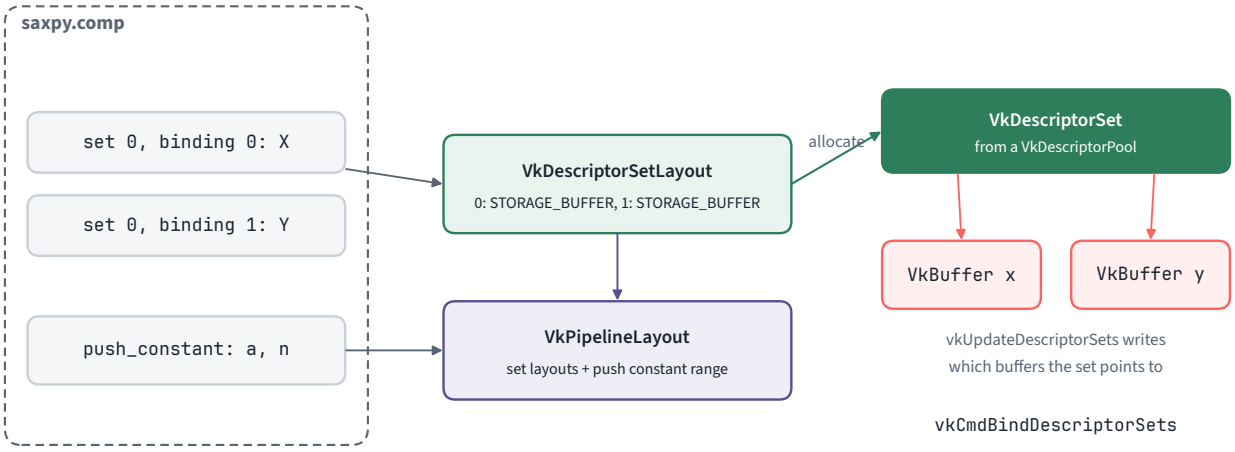


Figure 1.5.1. From the shader's bindings to real buffers. The set layout describes the bindings, and the pipeline layout collects set layouts and push constants. A set allocated from a pool is written to point at buffers and bound before the dispatch.

SAXPY in GLSL — code/u1/saxpy/saxpy.comp

```
#version 460

layout(local_size_x = 256) in;

layout(push_constant) uniform Push {
    float a;
    uint n;
} push;

layout(std430, set = 0, binding = 0) readonly buffer X { float x[]; };
layout(std430, set = 0, binding = 1) buffer Y { float y[]; };

void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i < push.n) {
        y[i] = push.a * x[i] + y[i];
    }
}
```

The descriptor set layout mirrors the bindings. A program usually creates its set layouts once, when it starts.

A set layout with two storage buffers — code/u1/saxpy/main.cpp

```
const VkDescriptorSetLayoutBinding bindings[] = {
    {.binding = 0,
     .descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
     .descriptorCount = 1,
     .stageFlags = VK_SHADER_STAGE_COMPUTE_BIT},
    {.binding = 1,
     .descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
     .descriptorCount = 1,
     .stageFlags = VK_SHADER_STAGE_COMPUTE_BIT},
};
const VkDescriptorSetLayoutCreateInfo setLayoutInfo{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
    .bindingCount = 2,
    .pBindings = bindings,
};
VkDescriptorSetLayout setLayout = VK_NULL_HANDLE;
U1_CHECK(vkCreateDescriptorSetLayout(device.device, &setLayoutInfo, nullptr, &setLayout));
```

Pools, sets and updates

Descriptor sets come from a pool that is created with a maximum number of sets and a budget of descriptors of each type. Allocation can fail with `VK_ERROR_OUT_OF_POOL_MEMORY` when either runs

out. Programs typically create one pool per frame or per subsystem and reset it whole with `vkResetDescriptorPool` instead of freeing sets one at a time. Freeing individual sets requires the flag `VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT`.

A pool with room for one set of two storage buffers, and the set — code/u1/saxpy/main.cpp

```
const VkDescriptorPoolSize poolSize{.type = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
                                    .descriptorCount = 2};
const VkDescriptorPoolCreateInfo poolInfo{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO,
    .maxSets = 1,
    .poolSizeCount = 1,
    .pPoolSizes = &poolSize,
};
VkDescriptorPool descriptorPool = VK_NULL_HANDLE;
U1_CHECK(vkCreateDescriptorPool(device.device, &poolInfo, nullptr, &descriptorPool));

const VkDescriptorSetAllocateInfo allocateInfo{
    .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO,
    .descriptorPool = descriptorPool,
    .descriptorSetCount = 1,
    .pSetLayouts = &setLayout,
};
VkDescriptorSet set = VK_NULL_HANDLE;
U1_CHECK(vkAllocateDescriptorSets(device.device, &allocateInfo, &set));
```

A newly allocated set points at nothing. `vkUpdateDescriptorSets` writes descriptors into it: for each binding, the buffer, the offset into it and the range the shader may see. Updating a set is a host operation that takes effect immediately. A set must therefore not be updated while a command buffer that uses it is pending, because the GPU may be reading it. Chapter 3.6 handles this with one set per frame in flight.

Pointing binding 0 at x and binding 1 at y — code/u1/saxpy/main.cpp

```
const VkDescriptorBufferInfo xInfo{
    .buffer = x.buffer, .offset = 0, .range = VK_WHOLE_SIZE};
const VkDescriptorBufferInfo yInfo{
    .buffer = y.buffer, .offset = 0, .range = VK_WHOLE_SIZE};
const VkWriteDescriptorSet writes[] = {
    {.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET,
     .dstSet = set,
     .dstBinding = 0,
     .descriptorCount = 1,
     .descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
     .pBufferInfo = &xInfo},
    {.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET,
     .dstSet = set,
     .dstBinding = 1,
     .descriptorCount = 1,
     .descriptorType = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
     .pBufferInfo = &yInfo},
};
vkUpdateDescriptorSets(device.device, 2, writes, 0, nullptr);
```

The buffers themselves are host-visible and persistently mapped, so the program fills them directly. On a discrete GPU, data read this often would be staged into device-local memory as Chapter 1.3 showed. Host-visible memory keeps a first program simpler.

Two buffers of n floats, filled through their mappings code/u1/saxpy/main.cpp

```
const VkDeviceSize bytes = VkDeviceSize(n) * sizeof(float);
const VkMemoryPropertyFlags hostMemory =
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT;
u1::Buffer x =
    u1::createBuffer(device, bytes, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT, hostMemory);
u1::Buffer y =
    u1::createBuffer(device, bytes, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT, hostMemory);
auto* xs = static_cast<float*>(x.mapped);
auto* ys = static_cast<float*>(y.mapped);
for (uint32_t i = 0; i < n; ++i) {
    xs[i] = float(i % 1000) * 0.25f;
    ys[i] = 1.0f;
}
```

Push constants and `std430`

The push constants are a `float` and a `uint`. The C++ structure must lay them out exactly as the shader expects.

The C++ mirror of the shader's push constant block code/u1/saxpy/main.cpp

```
struct Push {
    float a;
    uint32_t n;
};
```

Buffer and push constant blocks follow the `std430` layout rules, which the course always states explicitly:

- A scalar is aligned to its size: 4 bytes for `float`, `int` and `uint`.
- A `vec2` is aligned to 8 bytes; a `vec3` and a `vec4` to 16. A `vec3` therefore occupies 12 bytes but is followed by 4 bytes of padding unless a scalar fills them.
- An array's elements are aligned to their own alignment, and its stride is the element's size rounded up to that alignment.
- A structure is aligned to its most-aligned member, and its size is rounded up to that alignment.

Uniform buffer blocks follow `std140` by default, which differs in one respect that is easy to miss: array elements and structures are aligned to 16 bytes, so a `float data[4]` occupies 64 bytes instead of 16. A C++ mirror of either layout should state its offsets with `alignas` and check them with `static_assert`, because a mismatch produces no error and the shader reads the wrong bytes.

The pipeline layout and the pipeline are created as Chapter 1.4 described, with a push constant range of `sizeof(Push)`:

Pipeline layout and compute pipeline code/u1/saxpy/main.cpp

```
const VkPushConstantRange range{
    .stageFlags = VK_SHADER_STAGE_COMPUTE_BIT, .offset = 0, .size = sizeof(Push)};
const VkPipelineLayoutCreateInfo layoutInfo{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO,
    .setLayoutCount = 1,
    .pSetLayouts = &setLayout,
    .pushConstantRangeCount = 1,
    .pPushConstantRanges = &range,
};
VkPipelineLayout layout = VK_NULL_HANDLE;
U1_CHECK(vkCreatePipelineLayout(device.device, &layoutInfo, nullptr, &layout));
VkShaderModule module = u1::createShaderModule(
    device.device, u1::readSpirv(std::string(VKF_SHADER_DIR) + "/saxpy.comp.spv"));
const VkComputePipelineCreateInfo pipelineInfo{
    .sType = VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO,
    .stage = {.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
              .stage = VK_SHADER_STAGE_COMPUTE_BIT,
              .module = module,
              .pName = "main"},
    .layout = layout,
};
VkPipeline pipeline = VK_NULL_HANDLE;
U1_CHECK(vkCreateComputePipelines(device.device, VK_NULL_HANDLE, 1, &pipelineInfo, nullptr,
                                  &pipeline));
```

Recording the dispatch

The command buffer binds the pipeline, binds the descriptor set at set index 0 through the pipeline layout, writes the push constants and dispatches. `vkCmdDispatch` takes a number of workgroups in each of three dimensions; with 256 invocations per workgroup, `(n + 255) / 256` workgroups cover every element (Figure 1.5.2). Bindings and push constants stay in effect for later dispatches in the same command buffer, so the program then dispatches nine more times without binding anything again, reusing one descriptor set for all ten.

Each dispatch reads and writes `y`, which the previous one wrote, so consecutive dispatches have a read-after-write and a write-after-write dependency on the same buffer. A barrier from compute-shader writes to compute-shader reads and writes goes between them. After the last dispatch, a barrier from compute-shader writes to host reads prepares the results for the CPU, as the barrier after the copy did in Chapter 1.3.

Bind, push, dispatch ten times with barriers, and hand the results to the host code/u1/saxpy/main.cpp

```
const VkCommandBufferBeginInfo begin{
    .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
    .flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT,
};
U1_CHECK(vkBeginCommandBuffer(cmd, &begin));
vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, pipeline);
vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, layout, 0, 1, &set, 0,
                        nullptr);
const Push push{a, n};
```

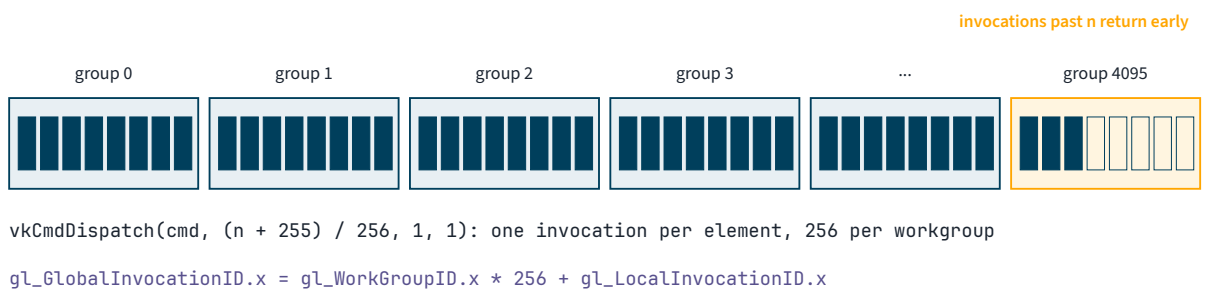


Figure 1.5.2. A one-dimensional dispatch. Every workgroup has 256 invocations. The last one has more invocations than elements, and the shader's bounds check stops them.

```
vkCmdPushConstants(cmd, layout, VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(push), &push);
const uint32_t groups = (n + 255) / 256;
for (int r = 0; r < repeats; ++r) {
    if (r > 0) {
        const VkMemoryBarrier2 between{
            .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
            .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
            .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
            .dstAccessMask =
                VK_ACCESS_2_SHADER_STORAGE_READ_BIT | VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        };
        const VkDependencyInfo dependency{.sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
                                          .memoryBarrierCount = 1,
                                          .pMemoryBarriers = &between};
        vkCmdPipelineBarrier2(cmd, &dependency);
    }
    vkCmdDispatch(cmd, groups, 1, 1);
}
const VkMemoryBarrier2 toHost{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_HOST_BIT,
    .dstAccessMask = VK_ACCESS_2_HOST_READ_BIT,
};
const VkDependencyInfo toHostDependency{.sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
                                        .memoryBarrierCount = 1,
                                        .pMemoryBarriers = &toHost};
vkCmdPipelineBarrier2(cmd, &toHostDependency);
U1_CHECK(vkEndCommandBuffer(cmd));
u1::submitAndWait(device, cmd);
```

`maxComputeWorkGroupCount` limits the number of workgroups per dimension. The guaranteed minimum is 65,535, which with 256-wide workgroups covers 16,776,960 elements; the M2 Pro allows 2^{30}. Chapter 2.6 handles larger problems.

> **KEY IDEA**
> A dispatch needs four things in place: a bound pipeline, the descriptor sets its layout names, the push constants it reads, and the barriers that order it against the work before and after it. The validation layer reports a missing pipeline or set, and reports a missing push constant only when the `maintenance4` feature is off, which `vkf` turns on wherever the device supports it. Only synchronisation validation reports a missing barrier.

Checking the answer

After ten dispatches every `y[i]` should equal `1 + 10 a x[i]`. The program checks every element against that formula in double precision and reports the largest relative error.

Checking every element on the CPU — code/u1/saxpy/main.cpp

```
double worst = 0;
for (uint32_t i = 0; i < n; ++i) {
    const double expected = 1.0 + repeats * double(a) * double(xs[i]);
    worst = std::max(worst, std::fabs(double(ys[i]) - expected) / expected);
}
const bool passed = worst < 1e-5;
std::printf("y = a*x + y, %d times: largest relative error %.2e (%s)\n", repeats, worst,
            passed ? "pass" : "FAIL");
```

SAXPY on an Apple M2 Pro

```
$ ./build/bin/u1_saxpy
Apple M2 Pro, n = 1048576
y = a*x + y, 10 times: largest relative error 0.00e+00 (pass)
```

The error is exactly zero here because every value involved is a multiple of 1/8 that a `float` represents exactly. With arbitrary inputs, expect differences of a few units in the last place, because the GPU may fuse the multiply and the add into one operation with a single rounding, and the CPU may not. Choose tolerances accordingly, and choose test data whose exact answer you know when you can.

> **COMING FROM OPENGL**
> OpenGL 4.3 added compute shaders. A program binds a buffer to an indexed binding point with `glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 1, buffer)`, sets parameters with `glUniform1f`, launches with `glDispatchCompute`, and calls `glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT)` before a later dispatch reads what an earlier one wrote. The binding points are global state of the context, and the next bind overwrites them. Vulkan's descriptor sets are objects that you prepare once and bind in one call. macOS stops at OpenGL 4.1, which has no compute shaders.

Reading

- The Vulkan specification, chapter "Resource Descriptors" (descriptor set layouts, pools, allocation, updates and binding) and the section "Dispatching Commands".
- The Vulkan specification, section "Offset and Stride Assignment" in "Shader Interfaces", which defines `std140`, `std430` and the scalar layout.
- The Vulkan Guide, "Shader Memory Layout", "Descriptor Dynamic Offset" and "Compute Shaders".

Labs

LAB 1.5.1 SAXPY end to end — 1.5 hours

Goal. Run, read and stress the complete program.

Code. `code/u1/saxpy`

1. Run `build/bin/u1_saxpy`, then read `main.cpp` from top to bottom and write one sentence in your notebook for each Vulkan call.
2. Run it with `--n` set to 1, 255, 256, 257 and 1,000,003.
3. Work out the largest `n` that your device's `maxComputeWorkGroupCount[0]` allows with 256-wide workgroups, and the largest that `maxStorageBufferRange` allows.

DONE WHEN

- [ ] Every run passes with no validation errors.
- [ ] You have the two limits for your device, and can say which one binds first.

EVIDENCE FOR THE NOTEBOOK

- The sentence-per-call annotation and the limits.

LAB 1.5.2 Vector addition — 1 hour

Goal. Write a second compute program from the first.

Code. `code/u1/saxpy`

1. Copy the project to `vadd`, add a third binding `z`, declared `writeonly`, and compute `z = x + y` once.
2. Extend the set layout, the pool and the descriptor writes for three buffers.
3. Check every element on the CPU.

DONE WHEN

- [ ] The program passes for `n` of 1 and 1,000,003, with no validation errors.

- [ ] Removing the third descriptor write produces a validation error at the dispatch, which you have recorded.

EVIDENCE FOR THE NOTEBOOK
- The differences from SAXPY, and the validation message.

LAB 1.5.3 Ping-pong between two sets — 1.5 hours

Goal. Reuse descriptor sets across dispatches.

Code. `code/u1/saxpy`

1. Allocate a second set that binds `y` at binding 0 and `x` at binding 1, so that a dispatch with it computes `x = a·y + x`.
2. Record ten dispatches that alternate between the two sets, with the barriers they need, and compute the expected values on the CPU.
3. Record which barriers you changed, and why the old ones are no longer enough.

DONE WHEN
- [ ] The program passes with synchronisation validation on.
- [ ] The pool is sized for exactly the sets and descriptors you allocate.

EVIDENCE FOR THE NOTEBOOK
- The recorded commands and your note on the barriers.

LAB 1.5.4 `std430` by hand — 1 hour

Goal. Make a C++ structure match a shader's layout exactly.

Code. `code/u1/saxpy`

1. Write a shader that reads an array of `struct Particle { vec3 position; float mass; vec2 velocity; uint id; }` from one buffer and writes each field, as floats, to a second buffer in a known order.
2. Write the C++ mirror with `alignas` where needed, and `static_assert` its size and every member's offset.
3. Fill the input with distinct values and check the output on the CPU.

DONE WHEN
- [ ] The offsets in your `static_assert`s are the `std430` offsets, and the check passes.
- [ ] You can state the structure's size and array stride, and what both would be in a `std140` uniform block.

EVIDENCE FOR THE NOTEBOOK
- The structure, the asserts and the two layouts compared.

Problem set

1. A shader uses set 0 with two storage buffers and set 1 with one uniform buffer. List every object the program must create to dispatch it, in order.
2. How many workgroups does `vkCmdDispatch` need for 1,000,003 elements with 64-wide workgroups? How many invocations do nothing?
3. Why may a program not call `vkUpdateDescriptorSets` on a set that a pending command buffer uses? What are two ways to change the buffers a shader sees between frames?

4. Give the `std430` and `std140` offsets of every member of `struct { float a; vec3 b; float c[3]; vec2 d; }`, and the structure's size under each.
5. The SAXPY program's error is exactly zero. Construct inputs for which the GPU's result differs from a naive C++ loop, and explain the difference.

COMPLETION CHECKLIST

- [ ] I can create descriptor set layouts, pools and sets, and write buffer descriptors into sets.
- [ ] I can lay out push constant and buffer data under `std430`, and mirror it in C++.
- [ ] I can record a dispatch with its bindings and the barriers before and after it.
- [ ] I can verify a GPU result on the CPU with a tolerance I can justify.
- [ ] Lab 1.5.1–1.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.6

Validation, debugging and a helper layer

TIME	5 hours
YOU BUILD	Six deliberately broken programs read through their messages; named objects; a capture of a compute dispatch; a shader that prints; and a small RAII layer of your own, compared with `vkf`, the helper library that Units 2 to 4 use.
NEEDS	Chapter 1.5. RenderDoc on Windows and Linux, or Xcode on macOS.

WHY IT MATTERS

SAXPY took about two hundred lines, and a mistake in almost any of them would have produced no error at all, because Vulkan does not check your calls and the GPU usually computes something plausible. This chapter is about the tools that find such mistakes, and about how to read what they report. It ends by wrapping the bookkeeping of Unit 1 in a small set of C++ helpers, so that from Unit 2 onwards the code shows what it computes, and every Vulkan call it makes is one you have already written by hand.

SKIP TEST

1. A message cites `VUID-vkCmdDispatch-None-08600`. What is a VUID, and where do you find the rule it names?
2. Name three kinds of mistake that the validation layer does not find in its default configuration.
3. What does `vkSetDebugUtilsObjectNameEXT` change in a validation message, and in a capture?
4. In what order must Vulkan objects be destroyed, and how can C++ destructors guarantee it?

If all four are easy, skim the core ideas and do Labs 1.6.1 and 1.6.4.

Core ideas

What the validation layer checks

`VK_LAYER_KHRONOS_validation` intercepts every Vulkan call and checks it against the specification's *valid usage* rules: thousands of statements of the form "parameter X must be Y". Its checks fall into groups that can be switched on and off separately:

- *Core validation* checks parameters, object state and the state recorded in command buffers: that descriptors match their layouts, that buffers have the usage flags their uses require, that a pipeline's layout matches the sets bound for it. *Object lifetime* checks catch handles used after destruction and objects still alive when their parent is destroyed. Both are on by default.
- *Thread safety* checks detect two threads using an externally synchronised object at once. The version of the layer used for this course disables them by default.
- *Synchronisation validation* (`validate_sync`) tracks every read and write of every resource and reports hazards that lack a barrier. It is off by default; Unit 1's programs and `vkf` turn it on.
- *GPU-assisted validation* (`gpuav_enable`) instruments your shaders so that, at run time on the GPU, out-of-bounds descriptor indexing, out-of-bounds accesses through buffer device addresses and invalid indirect parameters are reported. It is off by default and slow. At the time of writing it does not work through MoltenVK, because Metal fails to compile the instrumented shaders.
- *Debug printf* (`printf_enable`) lets shaders print, as described below.

- *Best practices* (`validate_best_practices`) warns about usage that is valid but likely slow, with optional checks for particular vendors.

A program can configure the layer in three ways. It can do so in code, through `VK_EXT_layer_settings`, as Chapter 1.1's instance creation does. Environment variables override that. Each setting has one named `VK_LAYER_` followed by the setting's name in capitals, such as `VK_LAYER_VALIDATE_BEST_PRACTICES=1` or `VK_LAYER_PRINTF_ENABLE=1`. Finally, the SDK's *Vulkan Configurator*, `vkconfig`, can force layers and settings onto any program on the machine, which is useful for programs you cannot change.

Validation slows every API call, often several times over, and synchronisation validation and GPU-assisted validation slow programs considerably more. Use them throughout development and in automated tests, and leave them out of releases and of performance measurements.

Reading a message

Every message has the same parts: a severity, the function that was called, a description of what was wrong, and a sentence that begins "The Vulkan spec states:", followed by a link that ends in a *VUID*, a valid-usage identifier. The VUID names the exact rule, and searching the specification for it shows the rule in context. The first of the chapter's broken programs creates its buffer for transfers only and then writes it into a storage-buffer descriptor:

The usage bug: a storage buffer created without VK_BUFFER_USAGE_STORAGE_BUFFER_BIT — code/u1/broken/main.cpp

```
const VkBufferUsageFlags usage =
    bug == "usage" ? VK_BUFFER_USAGE_TRANSFER_DST_BIT : VK_BUFFER_USAGE_STORAGE_BUFFER_BIT;
```

The message names the call, the descriptor, the flag the buffer lacks and the rule it breaks

```
$ ./build/bin/u1_broken --bug usage
bug: usage
[validation error] vkUpdateDescriptorSets(): pDescriptorWrites[0].pBufferInfo[0].buffer was
    created with VK_BUFFER_USAGE_2_TRANSFER_DST_BIT, but descriptorType is
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER.
The Vulkan spec states: If descriptorType is VK_DESCRIPTOR_TYPE_STORAGE_BUFFER or
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC, the buffer member of each element of
    pBufferInfo must have been created with the VK_BUFFER_USAGE_STORAGE_BUFFER_BIT usage
    flag set (https://docs.vulkan.org/spec/latest/chapters/descriptorsets.html#VUID-VkWriteD
    escriptorSet-descriptorType-00331)
validation reported 1 error(s)
values multiplied by four: yes
```

The program's own last line is the most instructive part of the output. The values were still multiplied correctly, because this GPU ignored the missing usage flag. Another GPU might place the buffer in memory a shader cannot reach, compress it in a format the shader cannot read, or fail in some other way. Invalid usage is undefined behaviour, so a correct result on one GPU says nothing about another.

Six bugs

`u1_broken` doubles an array of floats twice, with two dispatches. Its `--bug` option introduces one of six mistakes, each a line or two away from the correct program:

Option	Mistake and what reports it	Outcome on the M2 Pro
`usage`	Buffer created without storage usage; core validation at `vkUpdateDescriptorSets`.	correct
`descriptor-type`	A uniform-buffer descriptor written into a storage-buffer binding; two core validation errors.	correct
`push-range`	Eight bytes pushed into a four-byte push constant range; core validation at `vkCmdPushConstants`.	correct
`leak`	The buffer and its memory not destroyed before the device; object lifetime validation.	correct
`barrier`	No barrier between the two dispatches; synchronisation validation only.	correct
`unbound`	Dispatch with no descriptor set bound; core validation at `vkCmdDispatch`.	crash

Five of the six produced the right numbers. The missing barrier is the most dangerous of them, because only synchronisation validation detects it and its consequences depend on timing.

The barrier that `--bug barrier` leaves out — code/u1/broken/main.cpp

```
if (bug == "barrier") {
    const VkMemoryBarrier2 between{
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .dstAccessMask =
            VK_ACCESS_2_SHADER_STORAGE_READ_BIT | VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    };
    const VkDependencyInfo dependency{.sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
                                      .memoryBarrierCount = 1,
                                      .pMemoryBarriers = &between};
    vkCmdPipelineBarrier2(cmd, &dependency);
}
vkCmdDispatch(cmd, (n + 255) / 256, 1, 1);
```

A write-after-write hazard between two dispatches, found by synchronisation validation

```
$ ./build/bin/u1_broken --bug barrier
bug: barrier
[validation error] vkCmdDispatch(): WRITE_AFTER_WRITE hazard detected. Shader stage
    VK_SHADER_STAGE_COMPUTE_BIT writes to VkBuffer 0x20000000002[values], which was
    previously written by another vkCmdDispatch command.
The buffer is referenced by descriptor binding 0 (VK_DESCRIPTOR_TYPE_STORAGE_BUFFER) from
    descriptor set 0 (VkDescriptorSet 0x60000000006), VkPipeline 0x90000000009[double the
    values].
No sufficient synchronization is present to ensure that a write
    (VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT) at VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT does
    not conflict with a prior write of the same type at the same stage.
validation reported 1 error(s)
values multiplied by four: yes
```

The message names the hazard (`WRITE_AFTER_WRITE`), the buffer, the descriptor it was reached through, the pipeline, and the stage and access that the missing barrier needed to connect. The buffer and the pipeline appear as `[values]` and `[double the values]`, names the program gave them (see below), and the names make the message quick to act on.

The sixth bug shows why the layer's messages matter. The driver may assume valid usage, and here it did. After the layer reported the error twice, once for each dispatch, the program crashed inside the driver.

A dispatch with no descriptor set bound: the message, then a crash

```
$ ./build/bin/u1_broken --bug unbound
bug: unbound
[validation error] vkCmdDispatch(): The VkPipeline 0x90000000009[double the values]
    statically uses descriptor set 0, but because a descriptor was never bound, the
    VkPipelineLayouts are not compatible.
If using a descriptor, make sure to call one of vkCmdBindDescriptorSets,
    vkCmdPushDescriptorSet, vkCmdSetDescriptorBufferOffset, etc for
    VK_PIPELINE_BIND_POINT_COMPUTE.
The Vulkan spec states: If a a bound shader was created with a pipeline layout and
    statically uses a set n, a descriptor set must have been bound to n at the same pipeline
    bind point, with layouts compatible for set n with the layout(s) used to create the
    shader, as described in Pipeline Layout Compatibility (https://docs.vulkan.org/spec/late
    st/chapters/dispatch.html#VUID-vkCmdDispatch-None-08600)
[validation error] vkCmdDispatch(): The VkPipeline 0x90000000009[double the values]
...
zsh: segmentation fault  ./build/bin/u1_broken --bug unbound
```

> **KEY IDEA**
> Treat every validation error as seriously as a crash, because on another GPU or driver it may be one. The programs in this course fail when the layer reports an error, and yours should too, in every test run.

Names and labels

`VK_EXT_debug_utils` lets a program attach names to objects and labels to regions of a command buffer. Names appear in validation messages, in capture tools and in vendors' crash reports. Labels group commands in a capture, so that a frame of a thousand commands reads as a dozen named passes.

Naming an object, when the extension is available — code/u1/broken/main.cpp

```
void name(VkDevice device, VkObjectType type, uint64_t handle, const char* label) {
    if (setObjectName == nullptr) return;
    const VkDebugUtilsObjectNameInfoEXT info{
        .sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT,
        .objectType = type,
        .objectHandle = handle,
        .pObjectName = label,
    };
    setObjectName(device, &info);
}
```

The function is an extension function, looked up with `vkGetInstanceProcAddr` after the instance is created. Labels work the same way, through `vkCmdBeginDebugUtilsLabelEXT` and `vkCmdEndDebugUtilsLabelEXT`. Both cost little, so name every object you create.

Capturing a dispatch

A *capture* records every call a program makes, with the contents of its buffers and images, so that you can later inspect the state of the GPU before and after each command. RenderDoc, which is free and open source, captures Vulkan programs on Windows and Linux. It normally captures a frame between two presents. A compute program without a window marks the region to capture with RenderDoc's in-application API, by calling `StartFrameCapture` and `EndFrameCapture` from `renderdoc_app.h`. NVIDIA Nsight Graphics, AMD's Radeon GPU Profiler and Intel's Graphics Performance Analyzers add hardware counters, which Chapter 2.5 uses.

On macOS, MoltenVK can write a capture of the Metal commands it generates, which Xcode opens:

```
METAL_CAPTURE_ENABLED=1 MVK_CONFIG_AUTO_GPU_CAPTURE_SCOPE=1 \
  MVK_CONFIG_AUTO_GPU_CAPTURE_OUTPUT_FILE=saxpy.gputrace ./build/bin/u1_saxpy
```

The capture shows Metal's view of your program, including the Metal shader that MoltenVK generated from your SPIR-V, which is also where a name collision of the kind described in Chapter 1.4 becomes visible.

Printing from shaders

The `GL_EXT_debug_printf` extension adds `debugPrintfEXT` to GLSL. With debug printf enabled in the validation layer, the layer rewrites the shader so that each call writes its arguments into a buffer, and prints the results after the work completes:

```
#extension GL_EXT_debug_printf : enable
if (i < 4) debugPrintfEXT("x[%u] = %f\n", i, x[i]);
```

Enable it with `VK_LAYER_PRINTF_ENABLE=1`, and add `VK_LAYER_PRINTF_TO_STDOUT=1` to print to the terminal instead of through the debug messenger. To instrument the shader, the layer enables device features it needs, such as `bufferDeviceAddress`, and prints a warning that lists them. Without the layer, drivers ignore the call. Printing from thousands of invocations produces thousands of lines, so guard every call with a condition.

A helper layer

Most of SAXPY is bookkeeping: filling structures, checking results, and destroying everything in the right order. The order matters because every object must be destroyed before the object it was created from, and

the device and the instance last. C++ destroys local objects in reverse order of declaration, so a wrapper that destroys its handle in its destructor gets the order right automatically, provided the device's owner is declared first.

`vkf::Unique`: a move-only owner of one Vulkan handle — code/common/include/vkf/handles.hpp

```
template <class T>
class Unique {
public:
    Unique() = default;
    Unique(VkDevice device, T handle) : device_(device), handle_(handle) {}
    Unique(const Unique&) = delete;
    Unique& operator=(const Unique&) = delete;
    Unique(Unique&& other) noexcept
        : device_(other.device_), handle_(std::exchange(other.handle_, VK_NULL_HANDLE)) {}
    Unique& operator=(Unique&& other) noexcept {
        if (this != &other) {
            reset();
            device_ = other.device_;
            handle_ = std::exchange(other.handle_, VK_NULL_HANDLE);
        }
        return *this;
    }
    ~Unique() { reset(); }

    void reset() {
        if (handle_ != VK_NULL_HANDLE) {
            destroy(device_, handle_);
            handle_ = VK_NULL_HANDLE;
        }
    }
    T get() const { return handle_; }
    operator T() const { return handle_; }
    const T* ptr() const { return &handle_; }
    T release() { return std::exchange(handle_, VK_NULL_HANDLE); }
    explicit operator bool() const { return handle_ != VK_NULL_HANDLE; }

private:
    VkDevice device_ = VK_NULL_HANDLE;
    T handle_ = VK_NULL_HANDLE;
};
```

The course's helper library, `vkf`, is built from wrappers like this, and from functions that perform the steps of Unit 1 in one call each. Its central object is `vkf::Context`, which performs everything Chapter 1.1 did: instance, validation with synchronisation validation, the messenger, the device, its queues and a command pool. Its options say what a program needs.

What a program asks of the context — code/common/include/vkf/context.hpp

```
struct ContextOptions {
    std::string appName = "Vulkan course";
    // VKF_VALIDATION=0 and VKF_SYNC_VALIDATION=0 in the environment override these two.
    bool validation = true;
    bool syncValidation = true;
    // Ask for a second compute-capable queue, preferably in another family, for async compute.
    bool asyncCompute = false;
    // Ask for a queue in a transfer-only family when the device has one.
    bool transferQueue = false;
    std::vector<const char*> instanceExtensions;
    std::vector<const char*> deviceExtensions;
    // The main queue's family must be able to present to the returned surface.
    std::function<VkSurfaceKHR(VkInstance)> createSurface;
    bool quiet = false;
};
```

Memory is requested by intent instead of by flags, and `vkf` chooses the memory type as Chapter 1.2 described.

Four ways a buffer can be used, each mapped to memory flags — code/common/include/vkf/memory.hpp

```
enum class MemoryUse {
    DeviceLocal,  // fastest for the GPU; not mapped
    Upload,       // host-visible and coherent, written by the CPU and read by the GPU
    Readback,     // host-visible, and cached where the device offers it
    Shared,       // device-local and host-visible if available, otherwise like Upload
};
```

Table 1.6.1 maps each chapter of this unit to the `vkf` functions that replace its code.

Chapter	You wrote	vkf provides
1.1	instance, layers, messenger, device, queues	`Context`, `ContextOptions`, `mainQueue()`, `computeQueue()`, `transferQueue()`, `features()`, `properties()`
1.2	memory types, buffers, images, mapping, flushing	`createBuffer` with `MemoryUse`, `createImage`, `upload`, `download`, `Buffer::flush`, `Buffer::invalidate`
1.3	pools, command buffers, submission, fences, barriers	`submitNow`, `createCommandPool`, `submit`, `createFence`, `memoryBarrier`, `bufferBarrier`, `imageBarrier`
1.4	modules, layouts, pipelines, specialisation	`loadShader`, `shaderPath`, `createPipelineLayout`, `createComputePipeline`, `Specialization`
1.5	set layouts, pools, sets, updates	`DescriptorSetLayoutBuilder`, `DescriptorPool`, `DescriptorWriter`
1.6	names, labels, counting errors	`Context::name`, `beginLabel`, `endLabel`, `run`; and `GpuTimer` for Unit 2

Table 1.6.1. What each chapter of Unit 1 becomes in `vkf`

`vkf`'s self-test shows the result. The part below creates two buffers, uploads the input, builds a pipeline with two specialisation constants, writes a descriptor set and runs a dispatch with timestamps around it, in about fifty lines against SAXPY's two hundred.

A complete compute dispatch with vkf — code/common/test/selftest.cpp

```
std::vector<float> input(count);
std::iota(input.begin(), input.end(), 0.0f);
const VkDeviceSize bytes = count * sizeof(float);
vkf::Buffer src = vkf::createBuffer(
    ctx, bytes, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT,
    vkf::MemoryUse::DeviceLocal, "input");
vkf::Buffer dst = vkf::createBuffer(
    ctx, bytes, VK_BUFFER_USAGE_STORAGE_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    vkf::MemoryUse::DeviceLocal, "output");
vkf::upload(ctx, src, std::span<const float>(input));

auto setLayout =
    vkf::DescriptorSetLayoutBuilder()
        .add(0, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, VK_SHADER_STAGE_COMPUTE_BIT)
        .add(1, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, VK_SHADER_STAGE_COMPUTE_BIT)
        .build(ctx);
const VkPushConstantRange range{VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(ScalePush)};
auto layout = vkf::createPipelineLayout(ctx, {setLayout.get()}, {range});
auto module = vkf::loadShader(ctx, vkf::shaderPath("scale.comp"));
vkf::Specialization spec;
spec.set(0, uint32_t{256}).set(1, 0.5f);
auto pipeline = vkf::createComputePipeline(ctx, {.layout = layout,
                                                 .module = module,
                                                 .specialization = spec.info(),
                                                 .name = "scale"});

vkf::DescriptorPool pool(ctx);
VkDescriptorSet set = pool.allocate(setLayout, "scale");
vkf::DescriptorWriter()
    .buffer(0, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, src)
    .buffer(1, VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, dst)
    .update(ctx, set);

vkf::GpuTimer timer(ctx);
const ScalePush push{2.0f, count};
```

```
vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
    timer.reset(cmd);
    timer.stamp(cmd);
    ctx.beginLabel(cmd, "scale");
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, pipeline);
    vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, layout, 0, 1, &set, 0,
                            nullptr);
    vkCmdPushConstants(cmd, layout, VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(push),
                       &push);
    vkCmdDispatch(cmd, vkf::groupCount(count, 256), 1, 1);
    ctx.endLabel(cmd);
    timer.stamp(cmd);
});
const double ms = timer.elapsedMs(0, 1);
```

Every `vkf` program wraps its body in `vkf::run`, which catches exceptions, prints them, and returns a non-zero exit status if the body threw or if the validation layer reported any error. A program that exits with status 0 therefore had no validation errors.

`vkf` is deliberately thin. It never hides a barrier or a submission, and the functions that record them, such as `submitNow`, `upload` and the barrier helpers, say so in their names. Each of its functions does one thing that Unit 1 did by hand, so you can always read through it to the Vulkan calls underneath. It also makes simplifications that real programs should not: one allocation per buffer instead of sub-allocation, and `submitNow` and `upload`, which wait for the GPU before returning. Larger projects build on established libraries instead: Vulkan-Hpp, whose `vk::raii` namespace provides wrappers like `Unique` for every object; the Vulkan Memory Allocator; `vk-bootstrap` for instance and device creation; and `volk`, which loads Vulkan functions without going through the loader's dispatch.

> **COMING FROM OPENGL**
> An OpenGL error sets a flag that `glGetError` returns later, with a code such as `GL_INVALID_OPERATION` and no indication of which call or which rule. `KHR_debug` adds a callback with a message chosen by the driver. Capture tools such as RenderDoc support OpenGL too. OpenGL has no equivalent of synchronisation validation, because its driver inserts the barriers itself.

Reading

- The validation layer's documentation in the Vulkan-ValidationLayers repository on GitHub (`docs/` directory), in particular the pages on synchronisation validation, GPU-assisted validation and debug printf.
- The Vulkan Guide, "Vulkan Validation Overview" and "Development Tools".
- RenderDoc's documentation, "In-application API", for capturing programs without a window.

Labs

LAB 1.6.1 Read six broken programs — 1.5 hours

Goal. Learn to go from a message to a fix quickly.

Code. `code/u1/broken`

1. Run `build/bin/u1_broken --bug` with each of the six mistakes.
2. For each, record the VUID or hazard, explain the rule in your own words after reading it in the specification, and say whether the result was correct.
3. Fix each mistake in a copy of the program and confirm that the run is clean.

DONE WHEN

- [ ] Your table has six rows, each with the rule, the fix and the outcome.

- [ ] You can explain why the `unbound` case crashed and the others did not.

EVIDENCE FOR THE NOTEBOOK
- The table, and the crash explained in two sentences.

LAB 1.6.2 Print from a shader — 45 minutes

Goal. Look inside a running shader.

Code. `code/u1/saxpy`

1. Add the two lines of debug printf shown above to `saxpy.comp`, printing `x[i]` and `y[i]` before the update for the first four invocations.
2. Run with `VK_LAYER_PRINTF_ENABLE=1` and `VK_LAYER_PRINTF_TO_STDOUT=1`, and record the output and the warnings the layer prints at device creation.
3. Run again with `VK_LAYER_GPUAV_ENABLE=1` instead, and compare the run time with validation in its default configuration. On macOS, record the error that pipeline creation reports instead.

DONE WHEN
- [ ] Your notebook has the printed values for all ten dispatches and agrees with the CPU's arithmetic.
- [ ] You have the run times for default validation and debug printf, and for GPU-assisted validation or the error it produces on your platform.

EVIDENCE FOR THE NOTEBOOK
- The printed lines, the features the layer forced on, and the three run times.

LAB 1.6.3 Capture a dispatch — 1 hour

Goal. Inspect buffers before and after a dispatch in a capture tool.

Code. `code/u1/saxpy`

1. Name `x`, `y` and the pipeline, and wrap the dispatches in a label.
2. Capture one run: with RenderDoc's in-application API on Windows or Linux, or with the MoltenVK settings above on macOS.
3. In the capture, find the first dispatch, and the contents of `y` before and after it.

DONE WHEN
- [ ] Your capture shows the names and the label.
- [ ] You have recorded three values of `y` before and after the first dispatch, and they agree with the CPU's arithmetic.

EVIDENCE FOR THE NOTEBOOK
- A screenshot of the dispatch in the capture, with the buffer contents.

LAB 1.6.4 Your own RAII layer — 2 hours

Goal. Design the helpers yourself before using `vkf`.

Code. `code/u1/saxpy`

1. Write a move-only owner template for device-level handles and an owner for a buffer with its memory, without looking at `vkf`.
2. Rewrite SAXPY with them. Make every object an owner, and make the order of declarations guarantee the order of destruction.

3. Compare your design with `vkf/handles.hpp` and `vkf/memory.hpp`, and write down two differences and which design you prefer.

DONE WHEN

- [ ] Your SAXPY passes with validation on and no leaks reported, in fewer than 120 lines.
- [ ] Moving the device's owner after the buffers makes object lifetime validation report the error, which you have recorded.

EVIDENCE FOR THE NOTEBOOK

- Your helpers, the new SAXPY, and the comparison.

Problem set

1. Validation is on, synchronisation validation is on, and both are silent. Name three classes of bug that may remain, and a tool or technique that would find each.
2. Why do the validation layers offer settings at all, instead of running every check every time?
3. In the `unbound` case the driver crashed after the message. Why is it allowed to, and what does that imply for code that "works with validation errors"?
4. A `vkf::Unique<VkBuffer>` is declared before the `vkf::Context` whose device created the buffer. What happens at the end of the scope, and what does the validation layer report?
5. `vkf::createBuffer` makes one allocation per buffer. Using Chapter 1.2, explain when that would fail on a real device, and how you would change `vkf` to avoid it without changing its interface.

COMPLETION CHECKLIST

- [ ] I can configure the validation layer in code, with environment variables and with `vkconfig`, and say what each group of checks finds.
- [ ] I can read a validation message, find its rule by VUID, and fix the cause.
- [ ] I can name objects, label commands, capture a dispatch and print from a shader.
- [ ] I can explain how `vkf` maps to the Vulkan calls of Chapters 1.1 to 1.5.
- [ ] Lab 1.6.1–1.6.4 done-when criteria all hold, with evidence filed.

UNIT 1

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who writes GPU code for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
1.1	Instances, devices and queues	☐	☐	
1.2	Memory, buffers and images	☐	☐	
1.3	Commands and submission	☐	☐	
1.4	Shaders, SPIR-V and pipelines	☐	☐	
1.5	Descriptors and your first dispatch	☐	☐	
1.6	Validation, debugging and a helper layer	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ From memory, write the sequence of objects SAXPY creates, from the instance to the dispatch, with one sentence each on why it exists.
- ☐ Explain, for your own GPU, which memory type you would use for a staging buffer, for a buffer the GPU reads every frame, and for results read back by the CPU, citing your Lab 1.2.1 output.
- ☐ Draw the host and device timelines of one SAXPY run, marking where the barriers and the fence act and what each makes safe.
- ☐ Explain how a specialised workgroup size reaches the GPU, from the GLSL declaration through SPIR-V to `VkSpecializationInfo`.
- ☐ Given a validation message you have not seen before, find its rule in the specification and explain the fix.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____